

Review article

Twenty years of the Java Virtual Machine Tool Interface: A systematic literature review

Philipp Haindl^{ID}*, Patrick Kochberger^{ID}

University of Applied Sciences St. Pölten, Campus Platz 1, St. Pölten, 3100, Austria

ARTICLE INFO

Dataset link: <https://doi.org/10.5281/zenodo.19948192>

Keywords:

Java Virtual Machine Tool Interface (JVMTI)
 Java virtual machine
 Runtime monitoring
 Performance profiling
 Debugging

ABSTRACT

Context: The Java Virtual Machine Tool Interface (JVMTI) has been an essential part of the Java ecosystem for over two decades, enabling profiling, debugging, and runtime monitoring of Java applications. However, a substantial body of JVMTI-related research engages with JVMTI indirectly through derived frameworks such as DiSL and ShadowVM, rendering it partially invisible to keyword-based search.

Objective: To assess the state of research on JVMTI as both a directly used API and a substrate for derived frameworks, we conducted a systematic literature review following the guidelines of Kitchenham and Charters. **Methods:** We adopted a two-stream search strategy combining automated keyword-based search across five academic databases (Elsevier Scopus, dblp, IEEE Xplore, ACM Digital Library, SpringerLink) with forward snowballing seeded from the database-stream inclusions. After deduplication, eligibility assessment, conflict resolution, and four iterations of snowballing until saturation, 40 studies were selected: 24 from the database search and 16 from snowballing.

Results: We identify seven primary usage scenarios: debugging, performance profiling, security enforcement, fault injection, memory management, bytecode instrumentation, and JVMTI-based instrumentation frameworks as a cross-cutting category. *ClassFileLoadHook* emerges as the most referenced event, serving as the transport layer for load-time bytecode rewriting in both directly-coded agents and framework-based tools. Sixteen studies (40% of the corpus) use JVMTI indirectly through derived frameworks, a cluster that is inaccessible to keyword-based search and that reveals a coordinated post-2018 research program on parallel and concurrent JVM workload profiling.

Conclusion: JVMTI's scientific impact extends well beyond direct API usage, and a dual-stream search design is methodologically necessary for SLRs on software-infrastructure topics. We identify sampling-based event processing, dynamic event filtering, JIT-compilation-aware analysis, and bridging the academic-industrial gap as open avenues for future research.

Contents

1.	Introduction	2
1.1.	Background	2
1.1.1.	Agent architecture	3
1.1.2.	Event categories	3
1.1.3.	Function categories	3
1.1.4.	Known limitations	3
1.2.	Related surveys	3
1.3.	Contributions	4
2.	Methodology	4
2.1.	Research questions	4
2.2.	Search strategy	4
2.2.1.	Search method	4
2.2.2.	Search terms	4

* Corresponding author.

E-mail addresses: philipp.haindl@ustp.at (P. Haindl), patrick.kochberger@ustp.at (P. Kochberger).URLs: <https://research.ustp.at/en/researchers/philipp-haindl> (P. Haindl), <https://isf.ustp.at/en/team/patrick-kochberger> (P. Kochberger).

2.2.3.	Data sources	5
2.3.	Inclusion and exclusion criteria	5
2.4.	Study selection	5
2.4.1.	Identification	5
2.4.2.	Screening	5
2.4.3.	Eligibility	6
2.4.4.	Inclusion	6
2.5.	Forward snowballing	6
2.6.	Inter-rater agreement	7
2.7.	Study quality assessment	7
2.8.	Data extraction and synthesis	7
2.8.1.	Data extraction	7
2.8.2.	Synthesis	7
3.	Results	8
3.1.	Demographic attributes	8
3.1.1.	Studies distribution	8
3.1.2.	Research types	8
3.1.3.	Contribution types	8
3.1.4.	Publication venues	9
3.2.	Identified JVMTI events	9
3.3.	JVMTI usage scenarios	9
3.3.1.	Debugging and failure diagnosis	10
3.3.2.	Performance profiling and optimization	10
3.3.3.	Security enforcement and information flow control	11
3.3.4.	Fault injection and dependability analysis	12
3.3.5.	Memory management and garbage collection analysis	12
3.3.6.	Bytecode instrumentation and dynamic code analysis	12
3.3.7.	JVMTI-based instrumentation frameworks	13
4.	Discussion	13
4.1.	Dominant JVMTI events	13
4.2.	Dominant usage scenarios	13
4.3.	JVMTI as an ecosystem enabler	13
4.4.	Research trends	13
4.5.	Industrial relevance	14
4.6.	Limitations of the reviewed studies	14
4.7.	Future research directions	14
5.	Threats to validity	14
5.1.	External validity	14
5.2.	Internal validity	14
5.3.	Construct validity	14
5.4.	Conclusion validity	15
6.	Conclusion	15
	CRedit authorship contribution statement	15
	Declaration of competing interest	15
	Appendix A. List of selected studies	15
	Appendix B. Extraction and categorization schemes	17
	Appendix C. Classification of studies	17
	Data availability	17
	References	18

1. Introduction

The Java Virtual Machine Tool Interface (JVMTI) [1] is a critical component of the Java platform, providing a robust mechanism for profiling, debugging, and monitoring Java applications. Since its introduction over two decades ago, JVMTI has enabled developers and researchers to gain deep insights into program execution by facilitating access to low-level runtime events. As Java continues to be one of the most widely utilized programming languages, JVMTI remains relevant for optimizing performance, enhancing security, and enabling dynamic program analysis [2–5]. While JVMTI underpins widely-used industry profilers such as JProfiler, YourKit, VisualVM, and async-profiler, its academic exploration has remained comparatively niche, particularly in the context of dynamic program analysis and software monitoring.

Understanding how JVMTI has been explored in academic research is essential for identifying trends, potential applications, and gaps in the literature. Systematic Literature Reviews (SLRs) [6,7] are widely recognized as a rigorous method for synthesizing existing research,

ensuring a comprehensive, objective, and structured evaluation of a specific topic. Through the systematic collection, filtration, and analysis of published studies, an SLR provides valuable insights into research directions, commonly employed methodologies, and underexplored areas.

1.1. Background

JVMTI is a native programming interface designed to facilitate the monitoring, debugging, and performance analysis of Java applications at the Java Virtual Machine (JVM) level. Introduced with Java 5 in 2004 as the successor to the Java Virtual Machine Debug Interface (JVMDI) and the Java Virtual Machine Profiling Interface (JVMPPI) [8], it integrates profiling and debugging capabilities into a unified framework. JVMTI is part of the Java Platform Debugger Architecture (JPDA) and provides the lowest-level access to JVM state that is available to an external agent. The primary function of JVMTI is to provide a foundation for debugging, profiling, and monitoring tools that necessitate access to runtime data and execution events. This interface enables

Table 1

Comparison of related secondary studies on dynamic analysis, instrumentation, and JVM-related research.

Reference	Methodology	Year	# Studies	Scope	Distinction from this SLR
Cornelissen et al. [9]	SLR	2009	176	1999–2008	Surveys dynamic analysis as a research goal (program comprehension); classifies by analytical <i>purpose</i> rather than by the runtime substrate that enables it.
Durelli et al. [10]	SMS	2010	128	2003–2008	Maps research on high-level language virtual machines from a VM-construction perspective; JVMTI is referenced only incidentally and not examined as a research subject.
Maplesden et al. [11]	SMS	2015	253	2000–2013	Maps performance-analysis approaches for object-oriented software; JVMTI-based profilers are mentioned as tooling but are not the focus of analysis.
Chen and Jiang [12]	Survey	2021	69	1997–2019	Surveys source-level software log instrumentation; complementary perspective from the application-code side rather than from the JVM substrate.
Brant et al. [13]	Survey	2021	50	1980–2021	Reviews dynamic information-flow tracking; JVMTI is one of several DIFT mechanisms, organized by abstraction layer rather than by a JVM tooling API.
Sutter et al. [14]	SLR	2024	43	2017–2023	Reviews dynamic security analysis on Android; sister managed-runtime platform (Dalvik/ART) that lacks an analogous public tooling API.

developers to attach external agents to executing Java applications, facilitating detailed tracking of execution flow, memory management, and performance metrics. In contrast to conventional debugging tools, which operate at the source-code level, JVMTI affords more comprehensive insights into the JVM's internal operations by capturing method invocations, memory allocations, and garbage-collection events.

1.1.1. Agent architecture

A JVMTI agent is a native library (a .dll on Windows, a .so file on Linux and macOS) that is loaded into the JVM process at startup via the `-agentlib` or `-agentpath` command-line options. The agent registers for JVMTI *capabilities* and *event callbacks* at initialization time. Once loaded, the agent runs in the same process as the observed application but is logically isolated from it through the JVMTI interface boundary. Agents function as intermediaries that capture execution data and, in certain instances, modify program behavior through bytecode rewriting.

1.1.2. Event categories

JVMTI exposes approximately 40 event types, which can be grouped into six broad categories: (i) *class loading events* (*ClassFileLoadHook*, *ClassLoad*, *ClassPrepare*), enabling bytecode rewriting at load time; (ii) *thread lifecycle events* (*ThreadStart*, *ThreadEnd*), enabling thread monitoring; (iii) *method entry and exit events* (*MethodEntry*, *MethodExit*), supporting call-graph profiling; (iv) *memory management events* (*ObjectAlloc*, *VMOBJECTAlloc*, *GarbageCollectionStart*, *GarbageCollectionFinish*), supporting heap profiling; (v) *synchronization events* (*MonitorContendedEnter*, *MonitorContendedEntered*, *MonitorWait*), supporting lock-contention analysis; and (vi) *breakpoint, single-step, and exception events*, supporting interactive debugging tools.

1.1.3. Function categories

Beyond events, JVMTI provides functions for heap inspection (*IterateOverHeap*, *GetObjectsWithTags*), stack access (*GetStackTrace*, *GetAllStackTraces*), object tagging (*SetTag*, *GetTag*), bytecode rewriting (*RedefineClasses*), and thread control (*SuspendThread*, *ResumeThread*). The combination of event subscription and reflective function access is what makes JVMTI the most expressive observation interface available to external Java tooling.

1.1.4. Known limitations

JVMTI has several well-documented limitations that motivated the development of derived frameworks such as DiSL [15] and ShadowVM [16]: (a) *safepoint bias* in stack sampling, because *GetStackTrace* can only be called at JVM safepoints [17]; (b) *incomplete bytecode coverage* for classes loaded before the agent attaches; and (c) *in-process isolation issues*, because agents share heap and thread state with the observed application and can corrupt both observation and measurement [18]. The framework stream we review (see Section 3.3)

addresses these limitations through out-of-process instrumentation architectures that shift analysis to a separate JVM while retaining JVMTI as the underlying transport.

In conjunction with the Java Debug Interface (JDI) and the Java Debug Wire Protocol (JDWP), JVMTI forms the foundation of Java's debugging infrastructure: JDWP handles communication between a remote debugger and the JVM, while JDI provides a high-level Java API for interactive debugging tools.

1.2. Related surveys

To the best of our knowledge, no prior systematic literature review focuses specifically on the Java Virtual Machine Tool Interface (JVMTI) as its primary subject. Several adjacent surveys, however, examine research areas in which JVMTI features as one of multiple instrumentation mechanisms or as one of several runtime substrates. We position our work against the six most directly comparable secondary studies in Table 1.

The most directly adjacent prior work is Cornelissen et al. [9], who classified 176 studies on dynamic analysis for program comprehension. The two reviews share a methodological lineage but differ in their unit of analysis: Cornelissen et al. organize the literature by analytical *purpose* (program comprehension), whereas we organize it by technical *substrate* (JVMTI). The two surveys are thus complementary rather than overlapping. Chen and Jiang [12] similarly cover source-level log instrumentation, complementary to the JVM-level focus here.

A second cluster of related work surveys research at adjacent abstraction levels. Durelli et al. [10] scope JVM *construction* research rather than research *using* a specific JVM tooling API, while Maplesden et al. [11] cover performance analysis for object-oriented software without examining the underlying instrumentation APIs. Brant et al. [13] survey four decades of dynamic information flow tracking; one study in our corpus, Howarth et al. [19], falls within the scope of both surveys. Sutter et al. [14] review dynamic security analysis on Android, a sister managed-runtime platform that lacks an analogous public tooling API and instead relies on Xposed, Frida, and ART hooks; one paper in our corpus, Chang et al. [2], documents the porting of JVMTI semantics to Dalvik, illustrating that the same instrumentation problem space exists on both platforms.

Several broader secondary studies treat dynamic analysis or program analysis as a research field [20–24]. None examine JVMTI either directly or as a substrate, but together they document a methodological landscape in which mature secondary studies are organized by application domain or analytical method rather than by a single low-level runtime API.

The contribution of our review, viewed against this landscape, is twofold. First, we treat JVMTI as a first-class research subject and produce the first systematic account of its academic usage across two decades. Second, the dual-stream search design explicitly addresses an indirect-influence problem that is largely invisible to the surveys

above: when a technology is primarily accessed through derived frameworks, a keyword-only search systematically underestimates its research footprint. The methodological argument generalizes beyond JVMTI to other software-infrastructure topics (e.g., LLVM passes, eBPF programs, GraalVM Truffle interpreters) that have similar direct-versus-indirect usage structures.

1.3. Contributions

No prior systematic literature review specifically and comprehensively addresses JVMTI's use in academic research. Given JVMTI's role in low-level program analysis and the dispersed nature of relevant work across direct and indirect modes of use, a structured investigation is essential to identify usage trends, research gaps, and directions for future research. The main contributions of our work are as follows:

- C1. Dual-stream SLR methodology for infrastructure-level research topics:** We adopt a two-stream search strategy combining keyword-based database search with forward snowballing seeded from database-stream inclusions. The design addresses the indirect-influence problem of foundational software-infrastructure topics, which keyword-only search systematically underestimates. We demonstrate this point for JVMTI and document its generalization potential for related JVM-level infrastructure.
- C2. Bibliometric attributes of publications:** We analyze the distribution of 40 included studies over two decades by research type, contribution type, and venue, identifying a structural shift in research orientation after 2018.
- C3. Analysis of JVMTI events:** We examine which JVMTI events have been referenced or utilized across the corpus and identify *ClassFileLoadHook* as the single most influential event for both directly-coded agents and framework-based research.
- C4. Categorization of JVMTI usage scenarios:** We classify seven distinct usage areas, including the cross-cutting category of *JVMTI-based instrumentation frameworks*, and characterize how researchers leverage JVMTI both as an API and as a foundational substrate.
- C5. Characterization of JVMTI's indirect scientific impact:** Through forward snowballing, we identify and analyze 16 studies (40% of the final corpus) that use JVMTI indirectly through derived frameworks such as DiSL and ShadowVM. These studies are invisible to keyword-only search, and their identification reveals a coordinated post-2018 research program on parallel and concurrent JVM workload profiling that extends JVMTI's relevance well beyond the period of direct academic attention.

The subsequent sections of this paper are organized as follows: Section 2 elucidates the methodology employed for conducting the systematic literature review. Subsequently, Section 3 presents the results, before Section 4 discusses key findings, limitations of the current state of research, and future research directions. In Section 5, we elaborate on the potential threats to the validity of this study, before concluding the paper with final observations in Section 6.

2. Methodology

In the realm of Evidence-Based Software Engineering, a Systematic Literature Review (SLR) [6] is a widely recognized research methodology. It involves a methodical process [7] for identifying, evaluating, and synthesizing all pertinent evidence related to a particular research question or subject and consists of three primary phases: developing a review protocol, conducting the review, and reporting the findings.

We followed the guidelines outlined by Kitchenham and Charters [7], which encompassed: (i) formulating research questions, (ii) creating a search strategy, (iii) determining inclusion and exclusion criteria, (iv) choosing studies, and (v) extracting and synthesizing data.

To ensure transparency and reproducibility, we additionally adhered to the PRISMA 2020 guidelines [25].

2.1. Research questions

The objective of this research is to provide an overview of the current literature on JVMTI. To accomplish this, we have developed three research questions:

- RQ1.** What characterizes the research, contributions, publication channels, venues, and publication trends related to JVMTI?
- RQ2.** Which JVMTI events that an agent can be notified of at runtime are discussed in the studies?
- RQ3.** Which usage scenarios of JVMTI can be identified in the studies and how can they be categorized?

The first question characterizes the research landscape through bibliometric analysis of publication trends, contribution types, and venues. The second identifies which specific JVMTI events researchers have leveraged. The third classifies the identified usage scenarios into a structured taxonomy of JVMTI applications.

2.2. Search strategy

To maximize coverage, we adopted a two-stream search methodology. The primary stream is an automated database search following Kitchenham and Charters [7] and Zhang et al. [26]; the secondary stream is forward snowballing [27]. The forward stream is necessary because a substantial body of JVMTI-related work uses the API indirectly through derived frameworks such as DiSL [15] and ShadowVM [16]; such papers typically do not mention JVMTI in their title, abstract, or keywords and are thus invisible to keyword-based search.

2.2.1. Search method

Pilot searches were conducted in January 2025, with an iterative approach to refining the search string. In line with Zhang et al. [26], this refinement aimed, first, on ensuring the comprehensive inclusion of all relevant publications (i.e. *sensitivity*), and second, on minimizing the retrieval of irrelevant publications (i.e. *precision*). The primary database search for this study was conducted on 4 February 2025 across all five digital libraries on the same day to ensure comparability between retrieval results. The forward snowballing stream was performed between January and March 2026, using the database-stream included papers as seeds. The combined search and snowballing process covers publications from 2004 (the year of JVMTI's introduction with Java 5) up to and including March 2026.

2.2.2. Search terms

We followed the guidelines outlined by Zhang et al. [26] and systematically refined and evaluated our search terms to retrieve a suitable number of publications discussing JVMTI and its various applications. The construction of the search string was guided by the framework proposed by Petticrew and Roberts [28], incorporating elements related to *population* and *intervention* to ensure comprehensive coverage of relevant studies. To accurately capture relevant studies, we identified key search terms and expanded them using logical operators to construct a precise and effective search string. After multiple rounds of adjustments, we arrived at the final search string:

"jvmti" OR "jvm-ti" OR "jvm ti" OR "java virtual machine tool interface" OR "JVM Tool Interface"

Table 2

Query filters used for the digital libraries.

Library	Query filters
Elsevier Scopus	Document type: Conference paper, Articles and Reviews Subject areas: Computer Science, Engineering, and Decision Sciences Language: English
dblp	No filters
IEEE Xplore	Content type: Conferences, Journals
ACM Digital Library	Content type: Research Article
SpringerLink	Document type: Conference paper, Article, Research article Language: English

Table 3

Inclusion and exclusion criteria of this study.

<i>Temporal scope (both streams): publications from 2004 (JVMTI's introduction with Java 5) up to and including March 2026.</i>	
Inclusion criteria — database search stream	
I1	Publications elaborating on the usage of JVMTI
I2	Publications discussing specific JVMTI events that an agent can be notified of at runtime, including their practical applications or technical implications
I3	Publications addressing challenges, limitations, or trade-offs in using JVMTI
Exclusion criteria — title/abstract screening	
E1	Duplicates (same paper in multiple databases)
E2	Insufficient JVMTI focus based on abstract and keywords
E3	Publications in a language other than English
E4	Bachelor, Master or PhD thesis
E5	Non peer-reviewed work
E6	Off-topic domains
E7	Insufficient quality
Additional inclusion criteria — forward snowballing stream	
I4	The paper uses a framework documented in the literature as built on JVMTI (e.g., DiSL, ShadowVM, FRANC, Overseer) as a primary technical component
I5	The JVMTI-based framework is central to the paper's instrumentation, analysis, profiling, or security mechanism and not only superficially mentioned
I6	The paper investigates JVM bytecode instrumentation, dynamic analysis, or runtime monitoring where the framework abstracts the JVMTI layer
Additional exclusion criteria — full-text eligibility	
E8	Paper uses JVMDI or JPDA without any JVMTI dependency
E9	The cited seed paper appears only in the Related Work section
E10	The paper uses exclusively other instrumentation mechanisms
E11	The JVMTI-based framework is cited but not used practically

2.2.3. Data sources

We conducted searches in five major academic databases: *Elsevier Scopus*, *dblp* (computer science bibliography), *IEEE Xplore*, *ACM Digital Library*, and *SpringerLink*, chosen for their comprehensive coverage of scientific publications relevant to our research focus. Our initial search strategy targeted keywords, titles, and abstracts within these digital libraries. Upon realizing that full-text searches yielded additional pertinent articles, we expanded our approach to include entire document contents. In addition, we employed a set of search query filters (see [Table 2](#)) for each library, which reduced the retrieved publications to those most relevant to our study.

2.3. Inclusion and exclusion criteria

The articles obtained from the digital libraries were assessed against the predetermined inclusion and exclusion criteria listed in [Table 3](#). The criteria are organized in two groups: the first set (**I1–I3**, **E1–E7**) applies to the database search stream and assesses whether a publication directly discusses JVMTI; the second set (**I4–I6**, **E8–E11**) applies to the forward snowballing stream and assesses whether a publication uses JVMTI indirectly through a derived framework. A central design principle of the snowballing criteria is that the absence of the term “JVMTI” in a paper's title, abstract, or keywords is not a ground for exclusion: when a paper uses DiSL as its instrumentation layer, its authors typically write “we instrument using DiSL” rather than “we use JVMTI”, even though JVMTI remains the underlying mechanism.

Whereas the original review protocol used insufficient quality (**E7**) as a screening-stage exclusion criterion, the present review applies a

structured, post-screening quality assessment scheme (see [Section 2.7](#)); papers rated as low quality are retained in the corpus for descriptive completeness but excluded from cross-study synthesis claims.

2.4. Study selection

[Fig. 1](#) shows the stages of the study retrieval and selection process according to the PRISMA 2020 guidelines [25] for reporting systematic reviews.

2.4.1. Identification

A total of 608 records were returned by the database search queries across the five digital libraries (see [Table 4](#) for the per-library breakdown). Of these, 495 records were imported into the SLR management application; 113 records were excluded pre-import by automated checks (within-database duplicates identified by DOI matching, and records with insufficient metadata for reliable screening). After per-database deduplication during import, the imported records originated from Elsevier Scopus ($n = 206$), ACM Digital Library ($n = 168$), IEEE Xplore ($n = 64$), SpringerLink ($n = 56$), and dblp ($n = 1$).

2.4.2. Screening

The imported records were subjected to cross-database deduplication. 139 records identified as duplicates by DOI, title, and author matching were removed under **E1**, leaving 356 unique publications for title/abstract screening.

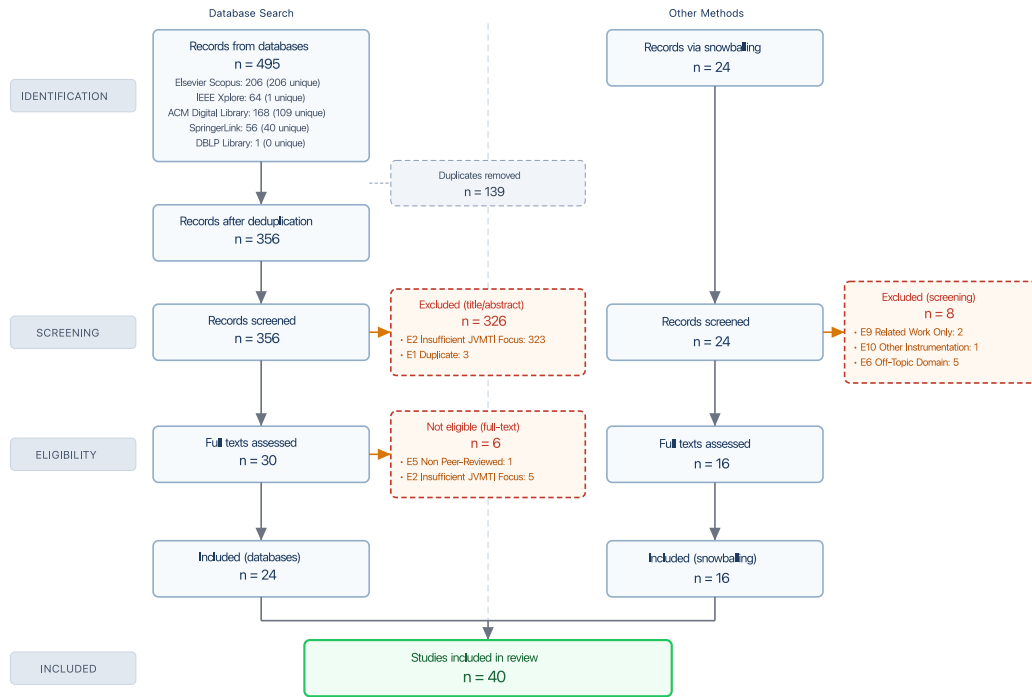


Fig. 1. Overview of the study retrieval and selection process according to PRISMA 2020 [25].

Table 4

Performance of the database search stream across the five digital libraries.

Library	Precision (%)	Recall (%)	F-score (%)
Elsevier Scopus	11.06 (24/217)	100.00 (24/24)	19.92
dblp	100.00 (1/1)	4.17 (1/24)	8.00
IEEE Xplore	6.79 (11/162)	45.83 (11/24)	11.83
ACM Digital Library	4.07 (7/172)	29.17 (7/24)	7.14
SpringerLink	1.79 (1/56)	4.17 (1/24)	2.50

2.4.3. Eligibility

We assessed 356 publications against the inclusion and exclusion criteria based on their title, abstract, and keywords. 326 publications were excluded at this stage: 323 for insufficient focus on JVMTI (E2) and 3 additional duplicates (E1) that had escaped automated deduplication. The remaining 30 publications advanced to full-text assessment, of which 6 were excluded (5 under E2 for insufficient JVMTI focus revealed only at full text, 1 under E5 for a non-recognized peer-review venue). Thus, 24 studies from the database search stream were included. An additional 16 studies were identified through forward snowballing (see Section 2.5), yielding a final corpus of 40 studies.

2.4.4. Inclusion

A total of 24 studies were included from the database search stream after retrieval, screening, and eligibility checks. Further studies were identified through forward snowballing (see Section 2.5). To evaluate the performance of the database search stream, we computed precision, recall, and F-score per database following Mourão et al. [29]. Precision counts unique retrievals per library as reported by the respective search interface; recall counts every retrieval of an included paper across libraries (cross-database duplicates contribute to each library's recall); F-score is the harmonic mean of the two.

Table 4 reports the resulting metrics for the five libraries against the 24 studies included via the database stream.¹

The results reveal a pronounced asymmetry in retrieval effectiveness. *Elsevier Scopus* is the only library that achieved full recall (24/24 included studies, precision 11.06% due to many keyword-matching non-relevant results). *IEEE Xplore* retrieved 11 of 24 (recall 45.83%) and *ACM Digital Library* 7 of 24 (recall 29.17%); *dblp* and *SpringerLink* identified one study each, with *dblp* achieving 100% precision due to its highly specific keyword match. Notably, every paper retrieved by a non-Scopus library was also retrievable through Scopus—a consequence of Scopus's broad indexing coverage of both IEEE and ACM venues. *F-scores* (Scopus 19.92%, IEEE Xplore 11.83%, *dblp* 8.00%, ACM 7.14%, SpringerLink 2.50%) confirm Scopus as the single most effective database for retrieving JVMTI-related literature; however, the sizeable contributions from IEEE Xplore and ACM, had Scopus been unavailable, demonstrate that multi-library retrieval provides substantial robustness against any single indexing gap. The search strings, filter settings, BibTeX exports, and search dates for all five libraries are documented in the replication package.

2.5. Forward snowballing

To complement the database search and capture studies that use JVMTI indirectly through derived frameworks, we applied forward snowballing [27] as a second search stream. Forward snowballing iteratively follows the citation chains of included studies to identify papers

¹ The supplementary review protocol additionally reports recall against the full 40-study corpus; the lower percentages there reflect the same numerators

against a denominator that includes the 16 snowballing-only studies, which by design are not retrievable through keyword-based search.

Table 5
Forward snowballing iterations.

Iteration	Candidates retrieved	Included	Saturated
1	9	7	No
2	8	6	No
3	4	3	No
4	3	0	Yes
Total	24	16	–

that cite them. As noted in the motivation for the dual-stream design (Section 2.3), a substantial body of JVMTI-related work engages with frameworks such as DiSL [15] and ShadowVM [16] rather than with the JVMTI API directly; such papers are invisible to keyword-based database search and require a citation-based retrieval strategy.

The snowballing process used the 24 database-stream included papers as seeds. For each seed, all citing publications were retrieved via Google Scholar and Semantic Scholar and screened against the inclusion criteria I4–I6 (see Table 3). Papers advancing to full-text assessment were reviewed against all applicable inclusion and exclusion criteria. The procedure was conducted iteratively, utilizing newly incorporated papers as the basis for each subsequent iteration, until saturation was achieved, defined as the point at which no new papers were identified in a given iteration.

Four iterations were performed between January and March 2026 (see Table 5). Iteration 1 retrieved 9 candidates from the seeds and included 7 of them; iteration 2 retrieved 8 candidates and included 6; iteration 3 retrieved 4 candidates and included 3. Iteration 4 retrieved 3 further candidates but yielded no new inclusions, confirming saturation. Across all iterations, 24 candidate records were assessed, of which 8 were excluded at title/abstract screening (5 under E6 for off-topic application domain, 2 under E9 for citing a seed paper only as related work, and 1 under E10 for the use of alternative instrumentation mechanisms) and 16 were included after full-text eligibility assessment. The 16 snowballing papers are assigned IDs S026–S041 in the corpus, bringing the final corpus to 40 studies. Inter-rater agreement for the snowballing stream is reported in Section 2.6.

2.6. Inter-rater agreement

To ensure the reliability of the screening process, both reviewers (R1 and R2) independently screened a quasi-gold-standard sample of 74 papers from the database stream. The sample was constructed by combining all papers initially flagged for inclusion by either reviewer with a stratified random sample of papers excluded by both, resulting in 74 papers covering 18 papers on which both reviewers agreed to include, 37 papers on which both reviewers agreed to exclude, and 19 papers on which the reviewers disagreed. This sampling strategy is consistent with established practice for SLRs in which included papers are substantially outnumbered by excluded papers [7].

Inter-rater agreement was quantified using Cohen's κ [30]. Given the prevalence asymmetry in the sample, we additionally report the Prevalence-Adjusted Bias-Adjusted Kappa (PABAK) [31], which corrects for both prevalence and reviewer bias. For the title/abstract screening stage, observed agreement was $P_o = 0.7432$, Cohen's $\kappa = 0.5091$ (95% CI: 0.3311–0.6871), and PABAK = 0.4865, indicating moderate agreement under the interpretation of Landis and Koch [32]. For title/abstract screening in a technical niche domain, this is consistent with values reported in the SLR literature [7].

The 19 disagreements were resolved through structured discussion under a four-step protocol: independent screening with rationale, classification on agreement, criterion-anchored discussion on divergence, and arbitration by a third reviewer if no consensus was reached. Arbitration was not invoked. After resolution, 7 papers advanced to full-text assessment and 12 remained excluded; the full conflict log with per-paper rationales is available in the replication package.

At the full-text eligibility stage, both reviewers independently reassessed the 46 papers advancing to full text (30 from the database stream, 16 from the snowballing stream). Observed agreement was $P_o = 1.0000$ and Cohen's $\kappa = 1.0000$, reflecting that the framework-based inclusion criteria I4–I6 and the full-text exclusion criteria E8–E11 are unambiguous for informed reviewers when applied to the full text. All screening decisions, rationales, conflict log, and agreement statistics are documented in the replication package.

2.7. Study quality assessment

Study quality was assessed using nine criteria adapted from Wieringa et al. [33] and Kitchenham and Charters [7], covering objective clarity, technical description of JVMTI usage, empirical evaluation, threats to validity, replication support, related-work comparison, contribution clarity, evaluation-subject appropriateness, and overhead measurement. Each criterion was scored 1 (fully met), 0.5 (partially met), or 0 (not met or not reported), for a maximum of 9.0 per paper. We define *High* ($\geq 6.75/9$), *Medium* ($\geq 4.5/9$), and *Low* ($< 4.5/9$) quality levels. Of the 40 studies, 28 were rated High, 9 Medium, and 3 Low (mean 80.6%). Low-quality papers are retained for descriptive completeness but excluded from cross-study synthesis claims [7].

Table A.7 in the Appendix provides a comprehensive list of all the studies that were ultimately selected for this review. The first column of the table serves as a unique identifier for each study, with the prefix “S” used to refer to the study within this paper. The following column indicates the title of the study, while the subsequent columns list the authors, venue, and year of publication for each study.

2.8. Data extraction and synthesis

2.8.1. Data extraction

Based on the properties delineated in Table B.8 in the Appendix, we extracted all requisite information from the selected studies to address the research questions of this SLR. A complete replication package (comprising all screening decisions with rationales, the inter-rater agreement and conflict-resolution logs, the forward-snowballing iteration logs, the full data-extraction records, the quality-assessment scores, and the ReviQ [34] project export) is openly available on Zenodo at <https://doi.org/10.5281/zenodo.19948192> under a CC-BY 4.0 licence.

2.8.2. Synthesis

The data extraction form has four sections. The first section captures bibliometric data (P1–P7); the second section extracts the JVMTI events referred to in a publication (P8), e.g., by reporting on an agent subscribing to particular JVMTI events; the third section captures the primary usage scenario presented in a publication (P9) together with reported challenges or limitations (P10); and the fourth section records the JVMTI usage mode (P11, direct or indirect) and, for indirect-use papers, the JVMTI-based framework(s) relied upon (P12). We applied descriptive statistics to analyze the properties P1–P5. These three attributes are orthogonal classification dimensions: a study's usage mode (P11), its primary usage scenario (P9), and its contribution type (P7) are assigned independently, so that, for instance, a study may use JVMTI indirectly through DiSL, be classified under performance profiling, and contribute a method.

To classify the research types (P6) of the selected papers, we employed six categories, namely *experience paper*, *evaluation research*, *validation research*, *philosophical paper*, *solution proposal* and *opinion paper*. These categories have been derived from the existing literature [33,35]. The definitions of each research type are provided in Table B.9 in the Appendix. In a similar vein, we extracted the contribution type (P7) of each paper's full text, which includes the following options: *model*, *method*, *theory*, *framework*, *guideline*, *lessons learned*, *advice* or *tool*. These options have been derived from the works of Shaw [36] and Paternoster

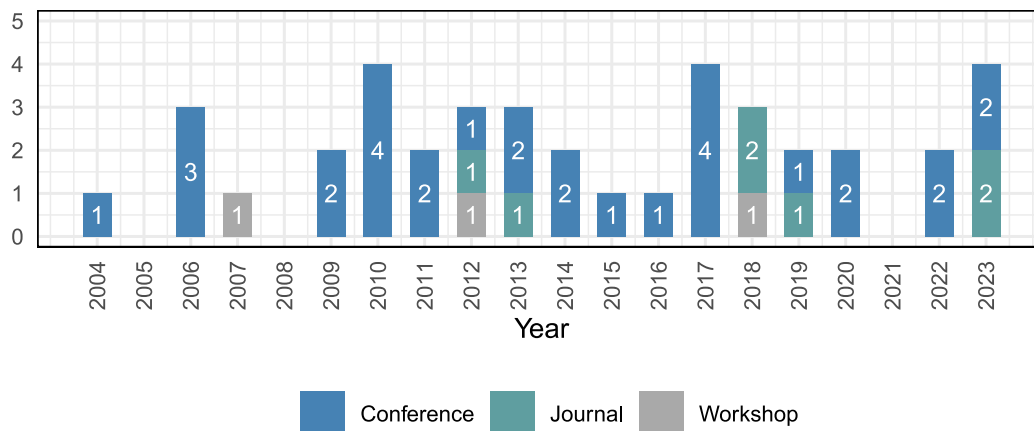


Fig. 2. Distribution of selected studies by year and publication type.

et al. [37]. The definitions of each contribution type are provided in Table B.10 in the Appendix.

JVMTI events (P8) were extracted by identifying explicit mentions in the full text of each publication; all referenced events were recorded. The primary usage scenario (P9) was assigned collaboratively: both reviewers independently classified each paper based on explicit and implicit indications, and discrepancies were resolved through discussion.

3. Results

The subsequent subsections present the findings derived from our examination and synthesis of data extracted from the reviewed literature to address the research questions. Our interpretations and reflections on these findings are provided in Section 4.

3.1. Demographic attributes

This section outlines the demographic characteristics of the examined studies, encompassing their chronological and publication-type distribution, as well as the nature of research conducted and its contributions. The information presented here addresses the first research question (RQ1) as outlined in Section 2.1. A comprehensive list of the selected papers and the data discussed in this section can be found in Table C.11 in the Appendix.

3.1.1. Studies distribution

We extracted the publication years of the analyzed studies to examine how research on JVMTI has evolved over time. As shown in Fig. 2, the selected studies were published between 2004 and 2023, spanning a period of 20 years. The earliest study in our dataset was published in 2004, indicating that JVMTI gained research attention soon after its introduction with Java 5. Publication activity exhibits two distinct phases: a first phase of sporadic, individual tool contributions between 2004 and 2018, with peaks of four publications each in 2010 and 2017; and a second phase from 2019 onwards characterized by a sustained research program of 10 studies (25% of the corpus), most of which focus on JVMTI-based frameworks for profiling parallel and concurrent Java workloads. In particular, 2023 marks another peak with four publications, underscoring that JVMTI-related research remains an active area rather than a closed topic.

With respect to publication types, conference papers constitute the majority of the included studies (31 of 40, 77.5%), indicating that JVMTI research is primarily disseminated through applied software-engineering and programming-language venues such as PPPJ, PLDI, OOPSLA, ISMM, ICPE, CGO, AOSD, and GPCE. Journals account for 15.0% of the corpus (6 studies), with publications appearing in ACM

TOPLAS, the *Journal of Network and Computer Applications* (JNCA), the *Journal of Visual Languages & Computing* (JVLIC), *Software: Practice and Experience* (SPE), *The Art, Science, and Engineering of Programming*, and *Control Engineering and Applied Informatics*. The remaining 7.5% (3 studies) appeared in specialized workshops (WODA, VMIL, BigBIA). The presence of multiple journal contributions, in particular in the post-2018 period, reflects the maturation of JVMTI-based research into long-form scholarly work.

3.1.2. Research types

According to Fig. 3(a), the corpus is dominated by two research types of approximately equal weight: *solution proposals* (17 studies, 42.5%) and *evaluation research* (16 studies, 40.0%). *Solution proposals* introduce new approaches to leveraging JVMTI, often providing proofs of concept or preliminary implementations but without extensive external validation; typical examples include the foundational proposals of DiSL [15] and ShadowVM [16], as well as domain-specific tooling such as P3 [38] and FJProf [39]. *Evaluation research* subjects existing artefacts or techniques to empirical study; this category includes studies such as the accurate-profiling evaluation of Zheng et al. [17], the task-granularity analyzes of Rosà et al. [40] and Rosà et al. [41], and the large-scale Java stream characterizations of Rosales et al. [42] and Rosales Rosero et al. [43].

Validation research contributes 15.0% of the corpus (6 studies), typically focusing on experimental validation of JVMTI-based mechanisms in controlled settings. The remaining 2.5% (1 study) is classified as *philosophical paper* [18], reflecting a critical examination of the conceptual foundations of JVM observability rather than an experimental contribution. No studies were classified as *experience papers* or *opinion papers*, indicating that reflections on industrial deployment or editorial commentary on JVMTI are absent from the peer-reviewed corpus.

The joint dominance of *solution proposals* and *evaluation research* reflects a healthy research cycle: the 2004–2018 period is rich in solution proposals introducing new JVMTI-based tooling, while the post-2018 period is dominated by evaluation research that systematically assesses those tools at scale, with a particular focus on the behavioral characterization of parallel and concurrent Java workloads.

3.1.3. Contribution types

In Fig. 3(b), we observe that *methods* constitute the largest share of contributions (15 of 40, 37.5%), closely followed by *tools* (14, 35.0%) and *frameworks* (10, 25.0%). Only one study (2.5%) contributes a *theory* [18]. No studies contribute *models*, *guidelines*, *lessons learned*, *advice*, or the remaining categories.

The sizeable *framework* share reflects the central role of reusable infrastructure in JVMTI research: ten studies contribute framework-level artefacts, including the foundational DiSL [15] and ShadowVM [16],

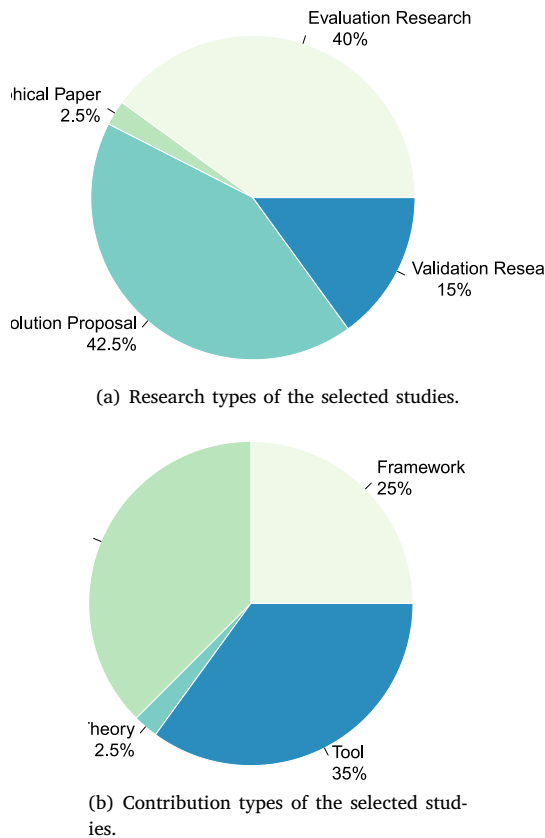


Fig. 3. Research and contribution types of the selected studies.

the JNIF bytecode instrumentation library [44], the BISM-based ShadowVM extension of Soueidi et al. [45], the Renaissance benchmark suite [46], and the Overseer hardware-counter bridge [47]. The prominence of *method* contributions in the post-2018 period — covering accurate profiling, supertype reification, task granularity analysis, and stream characterization — signals the transition from framework construction to framework-enabled methodological research, in which new analytical techniques are developed on top of the established JVMTI-based infrastructure.

3.1.4. Publication venues

The majority of publications appeared in conference proceedings. Venues hosting multiple JVMTI-related papers include the *International Conference on the Principles and Practice of Programming in Java (PPPJ)* and its successor *Programming on the Java Platform (PPPJ/ManLang)*, *GPCE*, *OOPSLA*, and the *ACM/SPEC International Conference on Performance Engineering (ICPE)*. Other notable conference venues include *PLDI*, *ISMM*, *AOSD*, *Euro-Par*, *APLAS*, *ARES*, *CGO*, and the *ICSE* satellite program (WODA).

Journal contributions are concentrated in the post-2015 period and include *ACM Transactions on Programming Languages and Systems (TOPLAS)* [41], *Software: Practice and Experience* [42], the *Journal of Visual Languages & Computing* [48], *The Art, Science, and Engineering of Programming* [43], the *Journal of Network and Computer Applications* [49], and *Control Engineering and Applied Informatics* [50]. The appearance of TOPLAS and SPE contributions underlines that JVMTI-based framework research has matured to a level that warrants long-form scholarly treatment in top-tier venues.

Our analysis further reveals that, outside of the clustered venues listed above, most individual venues hosted only a single JVMTI-related publication, highlighting that JVMTI research is widely dispersed across different forums rather than concentrated in a single

specialized outlet. This distribution suggests that while JVMTI remains relevant across software analysis, performance engineering, programming languages, and runtime research, its academic discourse cuts across disciplinary boundaries rather than clustering in a single specialized community.

3.2. Identified JVMTI events

To answer RQ2, we extracted the JVMTI events explicitly named in the analyzed studies. Table 6 summarizes the JVMTI events identified across the publications, reporting their frequency of reference and the studies that reference each event across the 40-study corpus.

The analysis reveals a clear three-tier hierarchy in event utilization across the corpus. The dominant event, referenced in 33 of 40 studies, is *ClassFileLoadHook*, which serves as the transport layer for load-time bytecode rewriting. This single event forms the foundation for a significant portion of direct-use studies, including the native-C++ JNIF framework [44], as well as the instrumentation approaches of Chapman et al. [51], Bergen and Edlich [52], and Wang and Kent [53], along with the Score-P tracing infrastructure of Frenzel et al. [54]. Additionally, it supports the entire snowballing cluster of framework-based papers, where DiSL [15] and ShadowVM [16] utilize class loading interception as their primary JVMTI mechanism, employing it as the conduit for out-of-process bytecode instrumentation. *ClassFileLoadHook*'s prominence reflects its role as the *de facto* foundation of JVMTI-based framework research.

The second tier comprises the method-execution events *MethodEntry* (13 studies) and *MethodExit* (12 studies), used in call-graph profiling, security-policy enforcement, and runtime monitoring [2,3,19,49,50,52,53,55–60]. The thread-lifecycle events *ThreadStart* and *ThreadEnd* (6 studies each) appear primarily in studies that track per-thread state for monitoring or migration [2,3,50,58,59,61], and *VMInit* (4 studies) serves as the agent-initialization hook in security and monitoring frameworks.

The third tier comprises specialized events. Memory-management events *ObjectAlloc* (3 studies), *ObjectFree* (3 studies), and *VMObjectAlloc* (2 studies), together with *GarbageCollectionStart* (4 studies) and *GarbageCollectionFinish* (1 study), appear in memory-profiling, supertype-tracking, and fork/join task-granularity work [16,40,48,60,62,63]. Synchronization events *MonitorContendedEnter*, *MonitorContendedEntered* (3 studies each), and *MonitorWait* appear in lock-contention profiling [64,65]. Field-access events (*FieldAccess*, *FieldModification*, 2 studies each) appear in information-flow-control and DTO-identification work [19,66]; *VMDeath* (2 studies) and *ClassLoad/ClassPrepare* (1 study each) serve as supporting primitives.

Interestingly, the corpus lacks events associated with just-in-time (JIT) compilation, such as *CompiledMethodLoad*, *CompiledMethodUnload*, and *DynamicCodeGenerated*. This absence is not incidental: Zheng et al. [17] explicitly analyze the challenges of profiling in the presence of dynamic compilation and propose a hybrid approach that combines bytecode instrumentation with JIT-aware adjustments, but the JVMTI JIT events themselves remain underused as a research object. Similarly, *SingleStep*, *FramePop*, *NativeMethodBind*, and *DataDumpRequest* do not appear in the reviewed literature, suggesting that fine-grained execution control, native-method resolution, and structured data collection through JVMTI have not been focal points of academic research.

3.3. JVMTI usage scenarios

In addressing RQ3, we examined the various applications of JVMTI as outlined in the chosen studies, exploring how it has been utilized across different research and practical settings. In the following, we present our results, organizing the observed usage scenarios into seven categories: debugging and failure diagnosis; performance profiling and optimization; security enforcement and information flow control; fault injection and dependability analysis; memory management and garbage collection analysis; bytecode instrumentation and dynamic code analysis; and JVMTI-based instrumentation frameworks as a cross-cutting category spanning the post-2012 corpus.

Table 6

JVMTI events referenced across the 40-study corpus, grouped by usage tier and ordered by frequency (S020 was excluded under E5).

JVMTI event	#	Studies
<i>Tier 1 — load-time transport</i>		
ClassFileLoadHook	33	S001, S003–S006, S009, S010, S012–S015, S017–S019, S021, S022, S024–S031, S033–S041
<i>Tier 2 — method execution and thread lifecycle</i>		
MethodEntry	13	S002–S004, S006, S007, S009–S011, S014, S016, S017, S022, S024
MethodExit	12	S002–S004, S007, S009–S011, S014, S016, S017, S022, S024
ThreadStart	6	S002, S007–S009, S011, S016
ThreadEnd	6	S002, S007–S009, S011, S016
VMInit	4	S007, S009, S011, S026
<i>Tier 3 — specialized events</i>		
GarbageCollectionStart	4	S012, S017, S027, S030
MonitorContendedEnter	3	S009, S023, S032
MonitorContendedEntered	3	S009, S023, S032
ObjectAlloc	3	S009, S012, S017
ObjectFree	3	S027, S029, S037
FieldAccess	2	S005, S010
FieldModification	2	S005, S010
VMDeath	2	S002, S009
VMOBJECTAlloc	2	S012, S017
GarbageCollectionFinish	1	S012
ClassLoad	1	S009
ClassPrepare	1	S009
MonitorWait	1	S023

Not observed in the corpus: the JIT-compilation events *CompiledMethodLoad*, *CompiledMethodUnload*, and *DynamicCodeGenerated*, and the execution-control events *SingleStep*, *FramePop*, *NativeMethodBind*, and *DataDumpRequest*.

3.3.1. Debugging and failure diagnosis

A significant portion of the analyzed studies focuses on leveraging JVMTI for debugging and failure diagnosis, demonstrating its capabilities in capturing runtime execution data, analyzing application behavior, and facilitating post-mortem debugging. These works highlight the importance of execution trace collection, method call monitoring, and thread state analysis in debugging complex Java-based applications, particularly in distributed systems and performance-sensitive environments.

Bergen and Edlich [52] propose a reference architecture for debugging large-scale data analytics systems (LSAs) by capturing runtime execution traces and replaying them in a localized debugging environment. Their approach isolates failures in distributed applications, such as Apache Flink, by extracting subsets of execution traces that can be replayed to diagnose errors efficiently. The system relies on JVMTI's thread-management functions, enabling developers to inspect and analyze the state of the application at the time of failure. While effective for identifying application-level failures, the approach does not focus on performance optimization.

Kaddour and Quinqueton [56] explore the use of JVMTI for dynamic application management through aspect-oriented programming (AOP) and Just-In-Time (JIT) compilation techniques. Their system utilizes programmable monitoring expressions, allowing developers to define customized runtime checks and interventions based on application behavior. By leveraging JVMTI's bytecode-instrumentation capabilities, their framework integrates dynamic monitoring with software components at runtime, enabling adaptive management of Java-based applications. The study evaluates the trade-offs between monitoring flexibility and performance overhead, demonstrating that while the approach enhances observability and control, further refinements are needed to optimize efficiency in distributed environments.

A similar focus on compilation-aware monitoring is presented by Kaddour and Quinqueton [55], where they propose a system that dynamically compiles monitoring expressions into machine code to improve runtime performance. By integrating AOP-based management strategies with JVMTI's monitoring features, they reduce the overhead typically associated with runtime monitoring. The system utilizes low-level JVMTI functions to track field modifications and program execution states, enabling a fine-grained approach to monitoring distributed

applications. However, the study notes that the system's tight coupling between high-level and low-level monitoring components requires further refinement to improve modularity and support for distributed monitoring.

Kell et al. [18] critically examine the observability limitations of the JVM and discuss how JVMTI can be leveraged to improve runtime introspection. They highlight shortcomings in existing JVM observation mechanisms, such as JPDA and JVMTI, and propose a hybrid approach combining internal and external observation techniques. Their work identifies fundamental challenges in JVM-based monitoring, including state corruption, deadlocks, and VM crashes, arguing that these issues stem from inherent design constraints rather than implementation flaws. While the study does not introduce a direct solution, it provides a strong argument for future improvements in JVM observability mechanisms and underscores the need for high-performance debugging frameworks. The limitations diagnosed by Kell et al. subsequently motivated the out-of-process instrumentation architectures of DiSL and ShadowVM discussed in Section 3.3.7.

3.3.2. Performance profiling and optimization

The largest usage scenario in the corpus, covering 16 of the 40 studies (40.0%), is performance profiling and optimization. Researchers leverage JVMTI's runtime-monitoring capabilities to analyze execution flow, detect performance bottlenecks, and optimize resource utilization. The studies in this category range from classical workload characterization to advanced parallel-workload profiling, covering Java workloads at multiple levels from native-code execution to garbage collection, lock contention, and fork/join task-granularity analysis.

Binder et al. [3] propose a quantitative evaluation of native-code execution in Java workloads, introducing a profiling agent based on JVMTI that monitors transitions between bytecode and native code execution. Their approach combines JVMTI events to track method invocations and execution times, allowing for precise measurements of native-code contributions to overall execution time. The study reveals that native-code execution accounts for 1%–20% of total execution time, emphasizing the need for optimized JNI (Java Native Interface) usage in performance-critical applications. While the profiling method demonstrates high accuracy with moderate overhead, its reliance on

JVMTI and bytecode instrumentation introduces a trade-off between precision and runtime impact, requiring further optimization.

Paternier et al. [47] introduce Overseer, a low-level hardware monitoring and management tool for Java, which utilizes JVMTI to access and analyze Hardware Performance Counters (HPCs) and Intelligent Platform Management Interface (IPMI) sensor data. The framework integrates JVMTI with C++-based libraries, exposing hardware-level performance data to Java applications. Overseer enables detailed runtime hardware-level insights, supporting performance optimizations such as thread affinity tuning and CPU-utilization tracking. The study concludes that Overseer provides real-time hardware monitoring for Java applications, though its applicability is currently limited to Linux environments, necessitating future extensions for broader platform support. Overseer subsequently became a component in several post-2018 task-granularity studies (see Section 3.3.7).

Rosà et al. [67] present a JVMTI-based adaptable runtime monitoring framework built on DiSL, which leverages dynamic instrumentation techniques to track JVM execution states and adjust monitoring intensity based on runtime conditions. Their approach aims to optimize performance-monitoring overhead by dynamically enabling or disabling specific JVMTI events. The study demonstrates that adaptive monitoring can significantly reduce overhead while maintaining detailed profiling accuracy, making it suitable for performance-sensitive Java applications. However, the authors note that managing dynamic changes in monitoring configurations introduces additional complexity, requiring careful balancing between observability and system impact.

Schörgenhuber et al. [64] extend the concept of runtime monitoring to lock-contention profiling, proposing a sampling-based approach to detect performance bottlenecks caused by thread-synchronization issues. Their framework, implemented using JVMTI, captures lock-contention events and analyzes their impact on execution performance, using events such as *MonitorContendedEnter*, *MonitorContendedEntered*, and *ClassFileLoadHook*. The study finds that JVMTI-based lock-contention profiling provides valuable insights into multithreaded performance issues, but also introduces additional profiling overhead. The authors suggest that future optimizations could focus on lightweight event-sampling techniques to minimize runtime interference while maintaining profiling accuracy. This line of work is continued by Liscano et al. [65], who build a classifier that maps JVMTI monitor-contention traces onto a set of known Java lock-contention anti-patterns, moving from raw measurement to automated diagnosis.

Wang and Kent [53] introduce a region-based approach to pipeline parallelism in Java programs, using JVMTI for dataflow analysis and execution partitioning. Their system detects data-communication patterns between different program regions, constructing a stream graph to optimize performance on multicore architectures. The tool identifies opportunities for pipeline parallelism, improving performance in multithreaded applications. While the results show significant performance improvements, the authors highlight that manual partitioning remains a challenge, and future work should aim to automate dataflow analysis for broader applicability.

The post-2018 performance-profiling literature is dominated by a coordinated research program built on DiSL and ShadowVM. Zheng et al. [17] analyze accuracy problems that arise when profiling Java workloads in the presence of JIT optimizations such as method inlining and stack allocation, and demonstrate that naive bytecode-level profilers can produce errors of 15%–45% in the presence of escape analysis. Rosà et al. [40] and the extended TOPLAS version [41] develop task-granularity analyzers for fork/join workloads and provide optimization guidance based on task-shape metrics. Rosà and Binder [38] and Rosales et al. [39] package these insights into reusable profiler suites; Basso et al. [68] subsequently refines fork/join profiling accuracy by systematically correcting for instrumentation-induced timing distortions. Three closely related studies [42,43,69] apply this instrumentation stack to large-scale Java stream workloads, demonstrating that JVMTI-based dynamic analysis can scale to corpora of thousands

of real-world projects mined from GitHub. The *Renaissance* benchmark suite [46] uses DiSL/ShadowVM to characterize parallel Java workloads and has become a reference benchmark in the JVM performance community.

3.3.3. Security enforcement and information flow control

Several studies in the analyzed corpus explore the use of JVMTI for security enforcement and information flow control, demonstrating its capabilities in runtime monitoring, policy enforcement, and tracking unauthorized access or data flow within Java applications. The works in this category leverage JVMTI's event-subscription mechanisms to detect security-relevant operations, enforce predefined security models, and monitor thread behavior to prevent malicious activities. These approaches illustrate the potential of JVMTI as a tool for strengthening Java application security, particularly in mobile-code execution, attestation frameworks, and dynamic information-flow control.

Fei et al. [59] propose a JVMTI-based framework for security-policy enforcement, integrating Model-Carrying Code (MCC) principles to ensure that Java applications adhere to predefined security policies at runtime. The system inspects and controls application behavior by intercepting and analyzing method calls, field accesses, and object allocations using JVMTI hooks. The framework employs JVMTI functions such as *GetThreadState*, *GetStackTrace*, *GetOwnedMonitorInfo*, and *SetEventCallbacks*, allowing fine-grained security-policy enforcement on a per-thread basis. Their results indicate that the framework effectively prevents unauthorized operations while maintaining acceptable performance overheads. However, they highlight that policy definitions must be carefully designed to avoid excessive runtime overhead and false positives in security-violation detection.

Gu et al. [49] introduce a security-model-oriented attestation scheme for dynamically reconfigurable component-based systems, leveraging Trusted Platform Module (TPM) technology in combination with JVMTI. The proposed system ensures that Java component execution adheres to predefined security models, using JVMTI to monitor runtime execution and verify the integrity of dynamically loaded components. The framework enables real-time attestation of component behavior, ensuring that applications remain compliant with security policies even in highly dynamic execution environments. While the results demonstrate practical feasibility, the approach's dependency on TPM hardware limits its general applicability, and further research is required to extend attestation mechanisms to non-TPM-based environments.

Howarth et al. [19] present a JVMTI-based information flow control (IFC) system, designed to monitor and restrict unauthorized information propagation within Java applications. Their system tracks both direct and indirect information flows at the level of primitive Java fields, ensuring that sensitive data is not leaked or improperly accessed. By leveraging JVMTI functions such as *SetFieldModificationWatch* and *SetFieldAccessWatch*, the framework dynamically analyzes data flow during program execution without requiring modifications to the JVM. The study demonstrates that the IFC system effectively detects and prevents unintended data flows but acknowledges that its applicability is currently limited to single-threaded applications, requiring future extensions for multi-threaded environments.

Sun et al. [58] propose a JVMTI-based model-enforcement mechanism for Model-Carrying Code (MCC), extending the concept of security enforcement through runtime policy validation. Their approach ensures that Java applications execute in accordance with predefined security models by monitoring and intercepting security-relevant method calls. The framework effectively enforces security constraints at the bytecode level, providing a portable security-enforcement solution that aligns with Java Specification Requests (JSRs). The evaluation results confirm that this approach provides robust security enforcement with minimal performance overhead, though the authors acknowledge that handling large-scale applications may require additional optimizations.

3.3.4. Fault injection and dependability analysis

Two studies leverage JVMTI for fault injection and dependability testing, aiming to evaluate system resilience by introducing controlled faults at runtime. These approaches demonstrate the effectiveness of JVMTI in simulating failure conditions, allowing researchers to analyze how Java applications respond to faults, measure fault-tolerance mechanisms, and improve system robustness. By utilizing JVMTI's capabilities for thread manipulation, exception triggering, and method interception, these studies provide insights into fault-tolerance strategies and dependability evaluation in Java-based systems.

Jacques-Silva et al. [70] introduce FIONA (Fault Injector Oriented to Network Applications), a JVMTI-based fault-injection framework for evaluating the dependability of Java-based network applications. The framework simulates faults such as omissions, delays, and message duplications, enabling researchers to assess the effectiveness of fault-tolerance mechanisms in communication protocols. By utilizing the JVMTI functions *SetEventNotificationMode*, *GetThreadState*, *SuspendThread*, *ResumeThread*, and *ForceGarbageCollection*, FIONA provides a low-overhead and portable solution for validating fault-tolerance strategies. The evaluation demonstrates the effectiveness of JVMTI-based fault injection in network environments, though the framework is currently limited to UDP-based applications, requiring extensions for broader protocol support such as TCP and Java Remote Method Invocation (RMI).

Silva et al. [57] propose a JVMTI-based Java fault injector for the Exhaustif Software Implemented Fault Injection (SWIFI) tool, designed to evaluate fault-tolerance mechanisms in Java applications. Their approach dynamically injects faults through bytecode instrumentation, simulating failures by corrupting method calls, substituting class implementations, and forcing unexpected exceptions. The fault injector seamlessly integrates into Java applications, allowing researchers to study failure propagation and recovery mechanisms. The evaluation confirms that the approach enhances interoperability with existing SWIFI tools while maintaining acceptable runtime overhead. However, the authors note that further optimizations are necessary to reduce performance impact, particularly in large-scale applications.

3.3.5. Memory management and garbage collection analysis

Two studies focus primarily on memory management and garbage collection (GC) analysis, tracking object allocation, monitoring garbage-collection events, and analyzing memory behavior at runtime. These studies leverage JVMTI's capabilities to provide insights into heap memory usage, object lifetimes, and GC efficiency, contributing to better understanding and optimization of Java memory-management strategies.

Ricci et al. [62] introduce *Elephant Tracks*, a JVMTI-based dynamic program analysis tool for precise garbage-collection event tracing. The framework captures object allocations, deaths, and pointer updates, allowing researchers to analyze memory-management efficiency and object lifecycles. By utilizing the JVMTI functions *FollowReferences*, *GetThreadState*, and *GetCurrentThreadCpuTime*, *Elephant Tracks* provides fine-grained insights into memory behavior. The study highlights that while the tool effectively tracks object deallocation and weak references, its performance overhead makes it less suitable for real-time applications.

In a follow-up study, Ricci et al. [60] extend *Elephant Tracks* to improve the completeness and precision of GC traces. Their enhanced approach utilizes JVMTI to track heap object allocations and garbage-collection events through *ClassFileLoadHook*, *GarbageCollectionFinish*, and *VMOBJECTAlloc*. The results demonstrate that the tool provides highly accurate object-lifecycle tracking, even for complex memory-allocation patterns. However, the authors acknowledge that the overhead associated with detailed GC tracing limits its applicability in performance-critical systems.

3.3.6. Bytecode instrumentation and dynamic code analysis

Several studies leverage JVMTI for bytecode instrumentation and dynamic code analysis, demonstrating its utility in modifying class-loading behavior, injecting runtime modifications, and analyzing application execution on the fly. These works highlight the effectiveness of JVMTI's *ClassFileLoadHook* and method-interception capabilities in dynamically altering Java program behavior, enabling use cases such as thread migration, transactional memory management, and tracing execution flows.

Chapman et al. [51] introduce XJ, a Java dialect that integrates transactional memory (TM) abstractions through bytecode instrumentation. Their approach employs JVMTI to intercept class loading and modify bytecode dynamically, injecting support for transactional memory operations without requiring modifications to the JVM itself. Their system enhances concurrency control and failure recovery in Java applications. While the approach significantly improves transactional execution efficiency, the authors acknowledge that manual effort in managing nested transactions remains a challenge.

Chang et al. [2] port the JVMTI specification to Google's Dalvik virtual machine, the runtime underlying early versions of Android. Their work implements the core JVMTI event and function set on top of Dalvik's register-based bytecode interpreter, enabling JVMTI-style debugging and profiling tools to operate on Android applications. The paper documents the engineering challenges of mapping JVMTI semantics onto a register-based VM that is architecturally distinct from the stack-based Oracle HotSpot, providing a useful reference for the portability characteristics of the JVMTI specification.

Dai et al. [61] propose a seamless service-migration framework using JVMTI, extending IBM's Service Management Framework (SMF) with a Mobile Services Framework (MoSeF) for Java thread migration. Their system leverages JVMTI to capture thread states, allowing services to migrate transparently across different nodes. The framework ensures thread persistency during migration, enabling stateful service relocation without service interruption. The study demonstrates that JVMTI facilitates finer-grained migration control, though handling running threads during migration remains a key challenge.

Frenzel et al. [54] explore JVMTI-based tracing for multi-threaded Java applications, particularly within Big Data frameworks like Apache Hadoop, Flink, and Spark. Their implementation enhances the Score-P tracing framework [71] by introducing automatic tracing using JVMTI and manual instrumentation via a Java API. Their approach provides detailed execution traces for performance analysis and debugging. However, the authors note that runtime filter checking can introduce significant overhead, limiting scalability in distributed systems.

Mastrangelo and Hauswirth [44] introduce JNIF (Java Native Instrumentation Framework), a C++ library for Java bytecode rewriting, enabling dynamic instrumentation within a JVMTI agent. Their system eliminates the need for an external JVM, reducing performance perturbations typically associated with bytecode-instrumentation frameworks. JNIF achieves efficient, low-overhead bytecode rewriting, outperforming traditional Java-based tools like ASM. However, limitations include incomplete support for *invokedynamic* instructions and potential issues with early class loading.

Morariu et al. [50] introduce a real-time monitoring solution for multi-tenant J2EE applications, integrating JVMTI with Java Authentication and Authorization Service (JAAS) and Enterprise Java Beans (EJB) interceptors. Their approach captures thread-execution events and authentication operations, linking them to tenant identities to provide low-overhead monitoring of enterprise applications. The system achieves precise CPU time and heap-space monitoring per tenant, though distinguishing tenant-specific from shared utility code remains a challenge.

Pantaleev and Rountev [66] propose a dynamic analysis technique for identifying Data Transfer Objects (DTOs) in Enterprise Java applications. Their method tracks object-field accesses and analyzes application-tier interactions to model DTO lifecycles and optimize

object-state management. The study demonstrates that the approach achieves high precision with minimal overhead, though further optimizations are needed to improve efficiency in large-scale enterprise applications.

3.3.7. JVMTI-based instrumentation frameworks

A seventh usage category, which cross-cuts the six preceding categories, comprises studies whose primary contribution is a framework that abstracts the JVMTI API for higher-level dynamic program analysis. Unlike papers in the *Bytecode instrumentation* category, these works treat JVMTI as an implementation substrate rather than as a directly used API, and their primary contributions are at the framework level rather than at the application level. This category was reached exclusively through the forward-snowballing stream and therefore constitutes the main finding enabled by the dual-stream search design.

The foundational works in this category are DiSL [15] and ShadowVM [16]. DiSL is a domain-specific language for bytecode instrumentation that uses JVMTI *ClassFileLoadHook* as its transport mechanism but performs instrumentation in a separate server process, overcoming the single-agent limitation that constrains purely in-process JVMTI agents. ShadowVM extends this design by running the analysis logic in a completely separate JVM instance that receives events over a message-passing interface, addressing the isolation and coverage limitations that Kell et al. [18] diagnose as fundamental to in-process observation. Together, DiSL and ShadowVM established the architectural template that shapes most subsequent JVMTI-based framework research.

Downstream works built on these frameworks span several subdomains. In parallel-workload profiling, Rosà and Binder [38] present the P3 profiler suite covering threads, tasks, actors, and synchronization primitives, while Rosales et al. [39] and Basso et al. [68] address fork/join profiling with a particular emphasis on accuracy under instrumentation-induced perturbation. In type-aware instrumentation, Rosà et al. [63] and Rosà and Binder [48] develop supertype-reification techniques that reduce the cost of type-specific interception. The Renaissance benchmark suite [46] integrates DiSL/ShadowVM for workload characterization and has become a reference benchmark in the JVM research community. In Java-stream analysis, Rosales et al. [42], Rosales et al. [69], and Rosales Rosero et al. [43] build on this stack to conduct large-scale empirical studies of stream usage across real-world Java codebases. Finally, Soueidi et al. [45] extend ShadowVM with the BISM instrumentation language and a complex-event-processing front end, enabling flexible specification of multi-event analyses.

4. Discussion

The systematic analysis of JVMTI-related research reveals several key insights regarding the usage scenarios, research focus, and limitations of the existing body of work. The identified studies demonstrate the broad applicability of JVMTI across debugging, performance profiling, security enforcement, fault injection, garbage collection analysis, dynamic bytecode instrumentation, and JVMTI-based instrumentation frameworks as a cross-cutting category. The 40-study corpus, identified through both database search and forward snowballing, is particularly illuminating in that it reveals a body of framework-based research whose indirect but foundational dependence on JVMTI has shaped JVM observability research since 2012. The following subsections highlight the dominant findings along with gaps and challenges that define the open research agenda.

4.1. Dominant JVMTI events

The single most frequently referenced event in the corpus is *ClassFileLoadHook*, the transport layer for load-time bytecode instrumentation in directly-coded JVMTI agents [44,70] and in framework-based tools [15–17,41], positioning it as the de facto foundation of JVMTI-based framework research. The second tier comprises method execution events (*MethodEntry*, *MethodExit*), thread lifecycle events (*ThreadStart*, *ThreadEnd*), *ClassLoad*, *VMObjectAlloc*, and the GC events *GarbageCollectionStart* and *GarbageCollectionFinish*. *MonitorContendedEnter* and *MonitorContendedEntered* appear prominently in lock-contention profiling [64,65]. JIT-related events (*CompiledMethodLoad*, *CompiledMethodUnload*, *DynamicCodeGenerated*) are largely absent, pointing to an open avenue for JIT-aware JVMTI research [17].

4.2. Dominant usage scenarios

Performance profiling and optimization is the largest category in the corpus, followed by bytecode instrumentation, security enforcement, debugging and failure diagnosis, fault injection, and memory management [3,18,19,44,47,49–62,64,66,67,70]. The seventh, cross-cutting category of *JVMTI-based instrumentation frameworks* [15,16,38,39,45,46,63,68] comprises papers whose primary contribution is a framework that abstracts the JVMTI API for higher-level dynamic program analysis. DiSL [15] and ShadowVM [16] are the foundational works; their descendants address parallel workload profiling [39,41,68], benchmarking infrastructure [46], Java stream characterization [42,43,69], and flexible dynamic analysis [45]. Such papers treat JVMTI as an implementation substrate rather than as a directly used API.

4.3. JVMTI as an ecosystem enabler

A key observation that emerges from the two-stream corpus is that JVMTI's scientific impact extends well beyond the 24 papers that use the API directly. The introduction of DiSL in 2012 [15] and ShadowVM in 2013 [16] — both built on JVMTI's *ClassFileLoadHook* mechanism — created a new layer of abstraction that enabled a generation of profiling and dynamic analysis tools whose authors engage with DiSL or ShadowVM rather than with JVMTI directly. The underlying API thus remains foundational but is no longer explicitly referenced in the publications that build on it.

The 16 papers identified through forward snowballing represent a substantial body of work — 40.0% of the final corpus — that would be invisible to a keyword-only search. Together, the 40 studies paint a picture of JVMTI not merely as an API but as a foundational infrastructure that has shaped JVM observability research across two decades. This finding has two methodological consequences. First, it indicates that systematic reviews of software-infrastructure topics must account for indirect influence through derived frameworks; a keyword-only search strategy would systematically underestimate the scientific footprint of any base technology that is commonly accessed through abstractions. Second, it invites future SLR work on other infrastructure-level technologies that are commonly accessed through abstractions to adopt similar dual-stream designs.

4.4. Research trends

Across all identified usage scenarios, several trends in JVMTI research can be observed. First, many studies employ JVMTI as part of larger frameworks or hybrid monitoring solutions, rather than using it in isolation. Research on debugging [52,56] and performance profiling [3,47] often integrates JVMTI with additional instrumentation techniques, such as bytecode rewriting, Just-In-Time (JIT) compilation modifications, or external profiling tools, highlighting the need for multi-layered introspection approaches.

Second, adaptive and selective monitoring approaches are becoming more relevant to address JVMTI's inherent performance overhead. Several studies attempt to dynamically enable or disable specific JVMTI events to optimize performance [64,67], yet scalability challenges persist in large-scale distributed systems. Future research must explore further optimizations, such as sampling-based event processing or integrating JVMTI with modern observability stacks like Extended Berkeley Packet Filter (eBPF) and OpenTelemetry.

Third, the corpus reveals a structural shift in research orientation after 2018. The post-2018 cluster is characterized by large-scale empirical studies particularly on Java streams [42,43,69] and by the maturation of JVMTI-based frameworks into stable research infrastructure used across multiple independent research groups. This suggests that JVMTI, through its framework abstractions, has transitioned from an API to an ecosystem enabler, as discussed in Section 4.3.

4.5. Industrial relevance

While this review focuses on academic publications as per its inclusion criteria, JVMTI also underpins production-grade industrial tools (JProfiler, YourKit, VisualVM, async-profiler), some of which combine JVMTI instrumentation with AsyncGetCallTrace to address the safepoint-bias problem documented in the academic literature [17,18]. These tools are excluded from the corpus because they do not produce peer-reviewed publications meeting inclusion criteria I1–I3, but their existence confirms that JVMTI's practical use extends beyond its peer-reviewed representation; bridging this gap through practitioner-oriented studies remains an open opportunity.

4.6. Limitations of the reviewed studies

Despite the versatility of JVMTI, the reviewed studies reveal several recurring limitations that constrain its practical adoption and effectiveness. A common concern across multiple categories is performance overhead. Studies on performance profiling and debugging acknowledge that continuous event monitoring, stack tracing, and bytecode instrumentation introduce runtime penalties that can distort the behavior of the monitored application [3,47,54,64]. While some works explore adaptive monitoring strategies to mitigate these effects, scalability in large-scale and distributed environments remains an open challenge. The snowballing cluster addresses this through out-of-process instrumentation [16], sampling-based approaches [64], and overhead-compensation techniques [43].

A second major limitation is manual configuration effort and usability concerns. Many studies, particularly those related to security enforcement and debugging, rely on manual policy definitions or static monitoring configurations, limiting their flexibility in handling diverse execution environments [19,58,59]. Moreover, dynamic bytecode instrumentation studies often report compatibility issues with specific JVM versions, JIT optimizations, and class loading mechanisms, raising concerns about portability and long-term maintainability [44,50,51]. The accurate-profiling challenges introduced by JIT optimizations such as stack allocation and method inlining are systematically documented by Zheng et al. [17], who report that on the DaCapo benchmark suite, the proportion of stack-allocated objects missed by naive bytecode-level allocation profilers reaches up to ~32% in individual workloads, with method inlining further compounding the error in callsite profiling.

Third, the lack of real-world validation and industrial adoption is apparent in multiple research categories. Many solution-proposal and validation-research studies provide proof-of-concept implementations, but only a few works conduct comprehensive evaluations in production-scale environments. The Java stream characterization studies [42,69] are notable exceptions: they employ large-scale GitHub mining to analyze stream usage across thousands of real-world projects, demonstrating that JVMTI-based dynamic analysis can scale to production codebases.

4.7. Future research directions

Several opportunities for future research emerge from the analyzed studies. First, improving the efficiency of JVMTI-based monitoring through lightweight, scalable solutions that integrate with modern observability frameworks remains an open problem; sampling-based profiling, dynamic event throttling, and hybrid instrumentation approaches could provide better trade-offs between performance and monitoring accuracy. Second, security policy enforcement is dominated by static policies and manual configuration; machine-learning-based anomaly detection, automated policy adaptation, and dynamic threat modeling could enhance JVMTI-based security frameworks, with containerized and microservice-based Java deployments as an underexplored target domain. Third, the post-2018 Java stream cluster [42,69] demonstrates that large-scale empirical studies using JVMTI-based dynamic analysis are feasible; future work should extend this approach to actor-based systems, microservices, and serverless Java deployments, and integrate JVMTI-based solutions with existing performance monitoring and debugging platforms.

5. Threats to validity

According to the taxonomy developed by Wohlin et al. [72], we identified potential threats to the validity of our systematic literature review, categorized into external, internal, construct, and conclusion validity.

5.1. External validity

External validity concerns the broader applicability of our findings beyond the selected studies [72]. The classification schemes proposed in this SLR (see Tables B.9 and B.10 in the Appendix) were derived from the reviewed literature; future studies may introduce additional categories or adjustments. The search window covers publications from 2004 (the introduction of JVMTI with Java 5) through March 2026; conclusions about the relative emphasis on individual usage scenarios are bounded by this window. The English-language restriction (E3) ensures consistent screening but may exclude relevant non-English work. Finally, the corpus is restricted to peer-reviewed academic publications; production-grade industrial tooling (briefly contextualized in Section 4.5) does not enter the corpus by design, and the findings therefore cannot be generalized to industrial JVMTI usage without further empirical evidence.

5.2. Internal validity

Internal validity relates to potential biases in study selection and data extraction [72]. Several steps in our research process could introduce bias. In the paper-selection process (see Section 2.4), inclusion and exclusion decisions involve a degree of subjectivity. To mitigate selection bias, two reviewers independently documented their inclusion and exclusion decisions based on the predefined criteria, inter-rater agreement was quantified with Cohen's κ and PABAK on a quasi-gold-standard sample (see Section 2.6), and any discrepancies were discussed and resolved through a formal conflict-resolution protocol. All screening decisions, rationales, and conflict-resolution outcomes are documented in the replication package, allowing the reader to audit individual inclusion decisions.

5.3. Construct validity

Construct validity concerns whether the study accurately captures the intended concepts [72]. A key threat lies in the search method (see Section 2.2.1): the risk of missing relevant publications due to terminology variations, indexing limitations, and the indirect-influence

problem (Section 2.2), where JVMTI-related work uses the API indirectly through derived frameworks and does not mention JVMTI in title, abstract, or keywords. To mitigate these risks, we systematically searched five major digital libraries (Section 2.2.3) and complemented keyword-based search with forward snowballing. The search string was constructed using the PICO framework [28] and iteratively refined based on pilot searches.

Forward snowballing scope. Forward snowballing was seeded from the 24 database-stream included papers and iterated until saturation over four iterations (Iteration 4 identified no new papers). It is possible that relevant work citing papers outside the seed set was not captured; the snowballing was conducted via Google Scholar and Semantic Scholar, which may have incomplete citation coverage for some venues. As a validation, we cross-checked Iteration 1 output against widely cited JVMTI-based framework papers known from the programming-languages community (DiSL [15], ShadowVM [16], the OOPSLA'15 accurate-profiling paper [17]); all were recovered organically in Iteration 1, providing evidence of adequate coverage.

5.4. Conclusion validity

Conclusion validity refers to the reliability of the results and the risk of misinterpretation [72]. The synthesis of extracted data, particularly the extraction of JVMTI usage scenarios, was subject to individual interpretation. To minimize bias, two reviewers independently reviewed each study, discrepancies were resolved through discussion, and the resulting data extraction and quality-assessment records were managed in the SLR management application. Papers rated Low quality (QA < 4.5/9; 3 studies) are reported descriptively but excluded from cross-study synthesis claims (see Section 2.7), reducing the risk that weakly supported findings influence aggregate conclusions. The complete screening decisions, conflict log, extraction records, and quality-assessment scores are available in the replication package to support re-analysis and scrutiny.

6. Conclusion

This systematic literature review of JVMTI identified 40 studies through a two-stream search (24 from database search, 16 from forward snowballing), with publication dates ranging from 2004 to 2023. The review reveals that JVMTI has sustained both direct use as a runtime API and indirect use as the foundation of an entire ecosystem of derived instrumentation frameworks, most prominently DiSL [15] and ShadowVM [16], which have amplified its scientific impact far beyond what keyword-based search alone suggests.

In the following, we summarize our main conclusions:

- (a) *ClassFileLoadHook* is the single most referenced JVMTI event across the corpus, serving as the transport layer for load-time bytecode rewriting in both directly-coded JVMTI agents (e.g., JNIF) and framework-based tools (DiSL, ShadowVM). Method execution events (*MethodEntry/MethodExit*), thread lifecycle events, and memory allocation/GC events form the second tier; JIT-related events remain largely absent, pointing to an open avenue for future JVMTI-based research.
- (b) JVMTI has been applied across seven main usage scenarios: debugging and failure diagnosis, performance profiling and optimization, security enforcement, fault injection, memory management, dynamic bytecode instrumentation, and the cross-cutting category of JVMTI-based instrumentation frameworks. Performance profiling dominates the corpus (40%), with bytecode instrumentation (17.5%) forming a substantial secondary cluster; the framework category spans 13 of the 40 studies (32.5%) and overlaps with the directly-classified categories rather than excluding them.

- (c) A structural shift is visible in the temporal distribution of the corpus. The post-2018 period comprises 10 studies (25% of the corpus) that form a coordinated research program on parallel and concurrent JVM workload profiling, built almost entirely on DiSL and ShadowVM. JVMTI's role has shifted from a directly used API to an enabling substrate for a productive family of framework-based research, contradicting the perception that JVMTI research has stagnated.
- (d) The corpus is dominated by *solution proposals* (42.5%) and *evaluation research* (40%), with *validation research* (15%) and a single *philosophical paper* (2.5%) completing the picture. The post-2018 prominence of evaluation research reflects the transition from framework construction to framework-enabled methodological research, in which new analytical techniques are systematically evaluated at scale.
- (e) Several recurring challenges were identified, including performance overhead from JVMTI-based instrumentation, safepoint bias in stack sampling, manual configuration complexity, and limited scalability in distributed systems. The limitations of directly-coded JVMTI agents (in-process isolation, incomplete bytecode coverage, cross-thread observability) explicitly motivated the out-of-process frameworks that dominate the post-2018 literature.
- (f) Among the 40 included studies, 24 use JVMTI directly as a runtime API, while 16 use JVMTI indirectly through a derived framework. The 16 papers identified through forward snowballing (40% of the corpus) are invisible to keyword-based search; their identification justifies the dual-stream search design and evidences JVMTI's role as a foundational infrastructure rather than merely an API.

Future research should focus on optimizing JVMTI-based monitoring by exploring techniques such as sampling-based event processing, dynamic event filtering, and hybrid instrumentation to minimize runtime overhead while maintaining detailed execution insights. Enhancing JVMTI's role in runtime security analysis through automated policy enforcement and anomaly detection could improve its practical applicability, especially in environments where security monitoring needs to be both efficient and adaptable.

Moreover, future studies should bridge the gap between academic research and industry adoption through empirical validations of JVMTI tools and JVMTI-based frameworks in enterprise settings, and through large-scale field studies that assess the practical benefits and limitations of JVMTI agents in real-world applications. The contrast between the academic corpus and industrial tools (JProfiler, YourKit, async-profiler, VisualVM, all of which use JVMTI internally) confirms that JVMTI's practical use extends beyond its peer-reviewed representation, and bridging this gap through practitioner-oriented studies remains an open opportunity.

CRediT authorship contribution statement

Philipp Haindl: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Data curation. **Patrick Kochberger:** Writing – review & editing, Validation, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. List of selected studies

See Table A.7.

Table A.7

Studies selected for this review.

Reference	Title	Venue	Year
Studies identified through the database search stream:			
[70]	FIONA: A Fault Injector for Dependability Evaluation of Java-Based Network Applications	IEEE International Symposium on Network Computing and Applications (NCA)	2004
[3]	A Quantitative Evaluation of the Contribution of Native Code to Java Workloads	IEEE International Symposium on Workload Characterization	2006
[55]	Proposal for Compilation Techniques of Monitoring Tasks to Improve Applications Management Performance	International Conference on Networking and Services (ICNS)	2006
[56]	Using Aspects and Compilation Techniques to Dynamically Manage Applications	International Conference on Security and Management (SAM)	2006
[66]	Identifying Data Transfer Objects in EJB Applications	International Workshop on Dynamic Analysis (WODA)	2007
[57]	Design and Implementation of a Java Fault Injector for Exhaustif [®] SWIFI Tool	International Conference on Dependability of Computer Systems (DepCos)	2009
[58]	JVMTI-based Model Enforcement on Java Platform for Model-Carrying Code	International Conference on Ubiquitous Information Technologies and Applications (ICUT)	2009
[2]	Implementation of JVM Tool Interface on Dalvik Virtual Machine	International Symposium on VLSI Design, Automation and Test (VLSI-DAT)	2010
[61]	A Seamless Services Migration Framework with JVM Tool Interface	International Conference on Computer Science and Education (ICCSE)	2010
[59]	JVMTI-Based Implementation of Security Policies Enforcement on Java Platform	International Conference on Advanced Computer Theory and Engineering (ICACTE)	2010
[19]	Information Flow Control Using the Java Virtual Machine Tool Interface (JVMTI)	International Conference on Availability, Reliability, and Security (ARES)	2010
[47]	Tool Demonstration: Overseer – Low-Level Hardware Monitoring and Management for Java	International Conference on the Principles and Practice of Programming in Java (PPPJ)	2011
[62]	Tool Demonstration: Elephant Tracks – Generating Program Traces with Object Death Records	International Conference on the Principles and Practice of Programming in Java (PPPJ)	2011
[49]	Security Model Oriented Attestation on Dynamically Reconfigurable Component-Based Systems	Journal of Network and Computer Applications	2012
[18]	The JVM is not Observable Enough (and What to Do About It)	ACM Workshop on Virtual Machines and Intermediate Languages (VMIL)	2012
[50]	Transparent Real Time Monitoring for Multi-Tenant J2EE Applications	Control Engineering and Applied Informatics	2013
[60]	Elephant Tracks: Portable Production of Complete and Precise GC Traces	International Symposium on Memory Management (ISMM)	2013
[51]	Closed and Open Nested Atomic Actions for Java: Language Design and Prototype Implementation	International Conference on Principles and Practices of Programming on the Java Platform (PPPJ)	2014
[44]	JNIF: Java Native Instrumentation Framework	International Conference on Principles and Practices of Programming on the Java Platform (PPPJ)	2014
[67]	Adaptable Runtime Monitoring for the Java Virtual Machine	International Symposium on Leveraging Applications of Formal Methods (ISoLA)	2016
[52]	Post-Debugging in Large-Scale Big Data Analytic Systems	Workshop on Big Data Management Systems in Business and Industrial Applications (BigBIA)	2017
[64]	Efficient Sampling-Based Lock Contention Profiling for Java	ACM/SPEC International Conference on Performance Engineering (ICPE)	2017
[53]	A Region-Based Approach to Pipeline Parallelism in Java Programs on Multicores	Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)	2017
[54]	Tracing of Multi-Threaded Java Applications in Score-P Using Bytecode Instrumentation	GI/ITG International Conference on Architecture of Computing Systems (ARCS)	2018
Studies identified through the forward snowballing stream (Iterations 1–3):			
[15]	DiSL: A Domain-Specific Language for Bytecode Instrumentation	International Conference on Aspect-Oriented Software Development (AOSD)	2012
[16]	ShadowVM: Robust and Comprehensive Dynamic Program Analysis for the Java Platform	International Conference on Generative Programming: Concepts and Experiences (GPCE)	2013
[17]	Accurate Profiling in the Presence of Dynamic Compilation	ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)	2015
[63]	Accurate Reification of Complete Supertype Information for Dynamic Analysis on the JVM	International Conference on Generative Programming: Concepts and Experiences (GPCE)	2017
[40]	Analyzing and Optimizing Task Granularity on the JVM	International Symposium on Code Generation and Optimization (CGO)	2018
[48]	Optimizing Type-Specific Instrumentation on the JVM with Reflective Supertype Information	Journal of Visual Languages and Computing	2018
[46]	Renaissance: Benchmarking Suite for Parallel Applications on the JVM	ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)	2019
[41]	Analysis and Optimization of Task Granularity on the Java Virtual Machine	ACM Transactions on Programming Languages and Systems (TOPLAS)	2019
[38]	P3: A Profiler Suite for Parallel Applications on the Java Virtual Machine	Asian Symposium on Programming Languages and Systems (APLAS)	2020
[39]	FJProf: Profiling Fork/Join Applications on the Java Virtual Machine	EAI International Conference on Performance Evaluation Methodologies and Tools (VALUETOOLS)	2020
[68]	Accurate Fork-Join Profiling on the Java Virtual Machine	European Conference on Parallel and Distributed Computing (Euro-Par)	2022
[69]	Characterizing Java Streams in the Wild	International Conference on Engineering of Complex Computer Systems (ICECCS)	2022

(continued on next page)

Table A.7 (continued).

[65]	A Lock Contention Classifier Based on Java Lock Contention Anti-Patterns	International Conference on Machine Learning and Applications (ICMLA)	2023
[42]	Large-Scale Characterization of Java Streams	Software: Practice and Experience	2023
[43]	Profiling and Optimizing Java Streams	The Art, Science, and Engineering of Programming	2023
[45]	Dynamic Program Analysis with Flexible Instrumentation and Complex Event Processing	IEEE International Symposium on Software Reliability Engineering (ISSRE)	2023

Table B.8

Properties extracted from the studies and related research questions.

#	Property	Description	RQs
P1	Title	Title of the paper.	RQ1
P2	Author(s)	Author(s) of the paper.	RQ1
P3	Year	Year of the paper.	RQ1
P4	Publication type	Type of the paper, i.e., journal article, conference paper, or workshop paper.	RQ1
P5	Venue	Name of the publication venue.	RQ1
P6	Research type	Type of research presented in paper (see Table B.9).	RQ1
P7	Contribution type	Type of contribution presented in paper (see Table B.10).	RQ1
P8	JVMTI event(s)	JVMTI event(s) referred to by a paper or subscribed to by a software agent.	RQ2
P9	Usage scenario	Primary usage scenario referred to or implemented by a software agent.	RQ3
P10	Challenges/limitations reported	Documented limitations of the proposed approach and of JVMTI as used in the paper (e.g., performance overhead, safepoint bias, isolation constraints).	RQ3
P11	JVMTI usage mode	Classifies whether the paper uses JVMTI directly (D) as a runtime API or indirectly (I) via a derived JVMTI-based framework such as DiSL or ShadowVM.	RQ1, RQ3
P12	Framework dependency	For indirect-use papers, the specific JVMTI-based framework(s) relied upon (e.g., DiSL, ShadowVM, JNIF, Overseer).	RQ3

Table B.9

Research types.

Source: Adapted from Petersen et al. [35], Wieringa et al. [33].

Research type	Description
Experience paper	Reports lessons learned from practical applications of JVMTI in debugging, profiling, or runtime monitoring, based on personal experiences rather than formal research methods.
Evaluation research	Analyzes the practical impact of JVMTI, assessing its advantages, limitations, and industry relevance through case studies, controlled experiments, or practitioner surveys.
Validation research	Investigates novel JVMTI-based applications that have not yet seen widespread adoption, using simulations, prototypes, or academic case studies.
Philosophical paper	Introduces a new conceptual framework or taxonomy related to JVMTI usage and categorization.
Solution proposal	Proposes a new approach to leveraging JVMTI, supported by a proof of concept but lacking full validation.
Opinion paper	Expresses personal views on JVMTI, evaluating their strengths and weaknesses without empirical evidence.

Table B.10

Contribution types.

Source: Adapted from Shaw [36], Paternoster et al. [37].

Contribution type	Description
Model	Abstract representation of JVMTI in a particular usage scenario.
Method	Specific approach or technique for utilizing JVMTI.
Theory	Explanation of cause-effect relationships regarding JVMTI's impact on JVM applications.
Framework	Combination of multiple JVMTI-based methods, defining applicable conditions, input parameters, and expected outcomes.
Guideline	Set of recommendations on best practices for using JVMTI, derived from research findings.
Lessons learned	Summary of findings and key takeaways from empirical studies on JVMTI applications.
Advice	General suggestions on JVMTI, based on personal opinions rather than empirical data.
Tool	Software implementation or technology leveraging JVMTI, e.g., as a JVM agent.

Appendix B. Extraction and categorization schemes

See Tables B.8–B.10.

Appendix C. Classification of studies

See Table C.11.

Data availability

A complete replication package (comprising all screening decisions with rationales, the inter-rater agreement and conflict-resolution logs, the forward-snowballing iteration logs, the full data-extraction records, the quality-assessment scores, and the ReviQ [34] project export) is openly available on Zenodo at <https://doi.org/10.5281/zenodo.19948192> under a CC-BY 4.0 licence.

Table C.11

Research and contribution type classification of the selected studies.

Reference	Research type	Contribution type
Studies identified through the database search stream:		
[70]	Validation research	Tool
[3]	Evaluation research	Method
[55]	Solution proposal	Method
[56]	Solution proposal	Framework
[66]	Validation research	Method
[57]	Solution proposal	Tool
[58]	Solution proposal	Method
[2]	Solution proposal	Tool
[61]	Solution proposal	Framework
[59]	Solution proposal	Method
[19]	Validation research	Tool
[47]	Validation research	Framework
[62]	Validation research	Tool
[49]	Evaluation research	Framework
[18]	Philosophical paper	Theory
[50]	Evaluation research	Method
[60]	Evaluation research	Tool
[51]	Solution proposal	Tool
[44]	Evaluation research	Framework
[67]	Validation research	Framework
[52]	Solution proposal	Method
[64]	Evaluation research	Tool
[53]	Solution proposal	Method
[54]	Evaluation research	Tool
Studies identified through the forward snowballing stream:		
[15]	Solution proposal	Framework
[16]	Solution proposal	Framework
[17]	Evaluation research	Method
[63]	Solution proposal	Method
[40]	Evaluation research	Tool
[48]	Solution proposal	Method
[46]	Evaluation research	Framework
[41]	Evaluation research	Tool
[38]	Solution proposal	Tool
[39]	Solution proposal	Tool
[68]	Evaluation research	Method
[69]	Evaluation research	Method
[65]	Evaluation research	Tool
[42]	Evaluation research	Method
[43]	Evaluation research	Method
[45]	Solution proposal	Framework

References

- [1] Oracle Corporation, JVM Tool Interface (JVM TI) Version 21.0.0, Oracle Technical Documentation, 2023, URL: <https://docs.oracle.com/en/java/javase/21/docs/specs/jvmti.html>. (Accessed 01 May 2026).
- [2] C.-W. Chang, C.-Y. Lin, C.-T. King, Y.-F. Chung, S.-Y. Tseng, Implementation of JVM tool interface on Dalvik virtual machine, in: Proceedings of 2010 International Symposium on VLSI Design, Automation and Test, 2010, pp. 143–146, <http://dx.doi.org/10.1109/VDAT.2010.5496711>.
- [3] W. Binder, J. Hulaas, P. Moret, A quantitative evaluation of the contribution of native code to Java workloads, in: 2006 IEEE International Symposium on Workload Characterization, 2006, pp. 201–209, <http://dx.doi.org/10.1109/IISWC.2006.302745>.
- [4] A. Gosain, G. Sharma, A survey of dynamic program analysis techniques and tools, in: S.C. Satapathy, B.N. Biswal, S.K. Udgata, J. Mandal (Eds.), Proceedings of the 3rd International Conference on Frontiers of Intelligent Computing: Theory and Applications (FICTA) 2014, Springer International Publishing, Cham, 2015, pp. 113–122.
- [5] B. Dufour, K. Driesen, L. Hendren, C. Verbrugge, Dynamic metrics for java, in: Proceedings of the 18th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA '03, Association for Computing Machinery, New York, NY, USA, 2003, pp. 149–168, <http://dx.doi.org/10.1145/949305.949320>.
- [6] B. Kitchenham, O. Pearl Brereton, D. Budgen, M. Turner, J. Bailey, S. Linkman, Systematic literature reviews in software engineering – A systematic literature review, Inf. Softw. Technol. 51 (1) (2009) 7–15, <http://dx.doi.org/10.1016/j.infsof.2008.09.009>.
- [7] B. Kitchenham, S. Charters, Guidelines for Performing Systematic Literature Reviews in Software Engineering, EBSE Technical Report EBSE-2007-001, Keele University and Durham University, 2007, URL: https://legacyfileshare.elsevier.com/promis_misc/525444systematicreviewsguide.pdf.
- [8] Oracle Corporation, Java Platform Debugger Architecture (JPDA), Oracle Technical Documentation, 2025, URL: <https://docs.oracle.com/javase/8/docs/technotes/guides/jpda/index.html>. (Accessed 01 May 2026).
- [9] B. Cornelissen, A. Zaidman, A. van Deursen, L. Moonen, R. Koschke, A systematic survey of program comprehension through dynamic analysis, IEEE Trans. Softw. Eng. 35 (5) (2009) 684–702, <http://dx.doi.org/10.1109/TSE.2009.28>.
- [10] V.H.S. Durelli, K.R. Felizardo, M.E. Delamaro, Systematic mapping study on high-level language virtual machines, in: Proceedings of the 4th Workshop on Virtual Machines and Intermediate Languages, VMIL'10, ACM, Reno, USA, 2010, pp. 1–10, <http://dx.doi.org/10.1145/1941054.1941056>.
- [11] D. Maplesden, E. Tempero, J. Hosking, J.C. Grundy, Performance analysis for object-oriented software: A systematic mapping, IEEE Trans. Softw. Eng. 41 (7) (2015) 691–710, <http://dx.doi.org/10.1109/TSE.2015.2396514>.
- [12] B. Chen, Z.M.J. Jiang, A survey of software log instrumentation, ACM Comput. Surv. 54 (4) (2021) 90, <http://dx.doi.org/10.1145/3448976>.
- [13] C. Brant, P. Shrestha, B. Mixon-Baca, K. Chen, S. Varlioglu, N. Elsayed, Y. Jin, J. Crandall, D. Oliveira, Challenges and opportunities for practical and effective dynamic information flow tracking, ACM Comput. Surv. 55 (1) (2021) 17, <http://dx.doi.org/10.1145/3483790>.
- [14] T. Sutter, T. Kehr, M. Rennhard, B. Tellenbach, J. Klein, Dynamic security analysis on Android: A systematic literature review, IEEE Access 12 (2024) 57261–57287, <http://dx.doi.org/10.1109/ACCESS.2024.3390612>.
- [15] L. Marek, A. Villazón, Y. Zheng, D. Ansaloni, W. Binder, Z. Qi, DiSL: a domain-specific language for bytecode instrumentation, in: Proceedings of the 11th Annual International Conference on Aspect-Oriented Software Development, AOSD '12, Association for Computing Machinery, New York, NY, USA, 2012, pp. 239–250, <http://dx.doi.org/10.1145/2162049.2162077>.
- [16] L. Marek, S. Kell, Y. Zheng, L. Bulej, W. Binder, P. Tůma, D. Ansaloni, A. Sarimbekov, A. Sewe, ShadowVM: robust and comprehensive dynamic program analysis for the java platform, in: Proceedings of the 12th International Conference on Generative Programming: Concepts & Experiences, GPCE '13, Association for Computing Machinery, New York, NY, USA, 2013, pp. 105–114, <http://dx.doi.org/10.1145/2517208.2517219>.
- [17] Y. Zheng, L. Bulej, W. Binder, Accurate profiling in the presence of dynamic compilation, SIGPLAN Not. 50 (10) (2015) 433–450, <http://dx.doi.org/10.1145/2858965.2814281>.
- [18] S. Kell, D. Ansaloni, W. Binder, L. Marek, The JVM is not observable enough (and what to do about it), in: 2012 ACM Workshop on Virtual Machines and Intermediate Languages, VMIL, 2012, pp. 33–38, <http://dx.doi.org/10.1145/2414740.2414747>.
- [19] J. Howarth, I. Altas, B. Dalgarno, Information flow control using the Java Virtual Machine Tool Interface (JVM TI), in: 5th International Conference on Availability, Reliability, and Security, ARES, 2010, pp. 689–695, <http://dx.doi.org/10.1109/ARES.2010.75>.
- [20] A. Tahir, S.G. MacDonell, A systematic mapping study on dynamic metrics and software quality, in: Proceedings of the 28th IEEE International Conference on Software Maintenance, ICSM 2012, IEEE Computer Society, Riva del Garda, Italy, 2012, pp. 326–335, <http://dx.doi.org/10.1109/ICSM.2012.6405289>.
- [21] X. Han, T. Yu, G. Yan, A systematic mapping study of software performance research, Softw.: Pract. Exp. 53 (6) (2023) 1249–1270, <http://dx.doi.org/10.1002/spe.3185>.
- [22] V.V. Kuliainin, A survey of software dynamic analysis methods, Program. Comput. Softw. 50 (1) (2024) 90–114, <http://dx.doi.org/10.1134/S0361768824010079>.
- [23] H. Cai, A survey of program analysis for distributed software systems, ACM Comput. Surv. 57 (12) (2025) 300, <http://dx.doi.org/10.1145/3742900>.
- [24] R. Taleb, S. Hallé, R. Khoury, Uncertainty in runtime verification: A survey, Comput. Sci. Rev. 50 (2023) 100594, <http://dx.doi.org/10.1016/j.cosrev.2023.100594>.
- [25] M.J. Page, J.E. McKenzie, P.M. Bossuyt, I. Boutron, T.C. Hoffmann, C.D. Mulrow, L. Shamseer, J.M. Tetzlaff, E.A. Akl, S.E. Brennan, R. Chou, J. Glanville, J.M. Grimshaw, A. Hróbjartsson, M.M. Lalu, T. Li, E.W. Loder, E. Mayo-Wilson, S. McDonald, L.A. McGuinness, L.A. Stewart, J. Thomas, A.C. Tricco, V.A. Welch, P. Whiting, D. Moher, The PRISMA 2020 statement: an updated guideline for reporting systematic reviews, Syst. Rev. 10 (1) (2021) 89, <http://dx.doi.org/10.1186/s13643-021-01626-4>.
- [26] H. Zhang, M.A. Babar, P. Tell, Identifying relevant studies in software engineering, Inf. Softw. Technol. 53 (6) (2011) 625–637, <http://dx.doi.org/10.1016/j.infsof.2010.12.010>.
- [27] C. Wohlin, Guidelines for snowballing in systematic literature studies and a replication in software engineering, in: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, EASE, ACM, 2014, pp. 38:1–38:10, <http://dx.doi.org/10.1145/2601248.2601268>.
- [28] M. Petticrew, H. Roberts, Systematic Reviews in the Social Sciences: A Practical Guide, first ed., Wiley-Blackwell, Malden, MA, 2005.
- [29] E. Mourão, J.F. Pimentel, L. Murta, M. Kalinowski, E. Mendes, C. Wohlin, On the performance of hybrid search strategies for systematic literature reviews in software engineering, Inf. Softw. Technol. 123 (2020) 106294, <http://dx.doi.org/10.1016/j.infsof.2020.106294>.
- [30] J. Cohen, A coefficient of agreement for nominal scales, Educ. Psychol. Meas. 20 (1) (1960) 37–46, <http://dx.doi.org/10.1177/00131646002000104>.

- [31] T. Byrt, J. Bishop, J.B. Carlin, Bias, prevalence and kappa, *J. Clin. Epidemiol.* 46 (5) (1993) 423–429, [http://dx.doi.org/10.1016/0895-4356\(93\)90018-V](http://dx.doi.org/10.1016/0895-4356(93)90018-V).
- [32] J.R. Landis, G.G. Koch, The measurement of observer agreement for categorical data, *Biometrics* 33 (1) (1977) 159–174, <http://dx.doi.org/10.2307/2529310>.
- [33] R. Wieringa, N. Maiden, N. Mead, C. Rolland, Requirements engineering paper classification and evaluation criteria: a proposal and a discussion, *Requir. Eng.* 11 (1) (2006) 102–107, <http://dx.doi.org/10.1007/s00766-005-0021-6>.
- [34] P. Haindl, ReviQ: A systematic literature review workbook, *SoftwareX* 35 (2026) 102814, <http://dx.doi.org/10.1016/j.softx.2026.102814>.
- [35] K. Petersen, S. Vakkalanka, L. Kuzniarz, Guidelines for conducting systematic mapping studies in software engineering: An update, *Inf. Softw. Technol.* 64 (2015) 1–18, <http://dx.doi.org/10.1016/j.infsof.2015.03.007>.
- [36] M. Shaw, Writing good software engineering research papers, in: 25th International Conference on Software Engineering, 2003. Proceedings, IEEE, 2003, pp. 726–736, <http://dx.doi.org/10.1109/ICSE.2003.1201262>.
- [37] N. Paternoster, C. Giardino, M. Unterkalmsteiner, T. Gorschek, P. Abrahamsson, Software development in startup companies: A systematic mapping study, *Inf. Softw. Technol.* 56 (10) (2014) 1200–1218, <http://dx.doi.org/10.1016/j.infsof.2014.04.014>.
- [38] A. Rosà, W. Binder, P3: A profiler suite for parallel applications on the Java virtual machine, in: B.C.d.S. Oliveira (Ed.), *Programming Languages and Systems, Springer International Publishing, Cham*, 2020, pp. 364–372.
- [39] E. Rosales, A. Rosà, W. Binder, FJProf: Profiling fork/join applications on the Java virtual machine, in: Proceedings of the 13th EAI International Conference on Performance Evaluation Methodologies and Tools, VALUETOOLS '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 128–135, <http://dx.doi.org/10.1145/3388831.3388851>.
- [40] A. Rosà, E. Rosales, W. Binder, Analyzing and optimizing task granularity on the JVM, in: Proceedings of the 2018 International Symposium on Code Generation and Optimization, CGO '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 27–37, <http://dx.doi.org/10.1145/3168828>.
- [41] A. Rosà, E. Rosales, W. Binder, Analysis and optimization of task granularity on the Java virtual machine, *ACM Trans. Program. Lang. Syst.* 41 (3) (2019) <http://dx.doi.org/10.1145/3338497>.
- [42] E. Rosales, M. Basso, A. Rosà, W. Binder, Large-scale characterization of Java streams, *Softw.: Pract. Exp.* 53 (9) (2023) 1763–1792, <http://dx.doi.org/10.1002/spe.3213>.
- [43] E. Rosales Rosero, M. Basso, A. Rosà, W. Binder, Profiling and optimizing Java streams, *Art. Sci. Eng. Program.* 7 (2023) <http://dx.doi.org/10.22152/programming-journal.org/2023/7/10>.
- [44] L. Mastrangelo, M. Hauswirth, JNIF: Java native instrumentation framework, in: Proceedings of the 2014 International Conference on Principles and Practices of Programming on the Java Platform: Virtual Machines, Languages, and Tools, PPPJ '14, Association for Computing Machinery, New York, NY, USA, 2014, pp. 194–199, <http://dx.doi.org/10.1145/2647508.2647516>.
- [45] C. Soueidi, Y. Falcone, S. Hallé, Dynamic program analysis with flexible instrumentation and complex event processing, in: 2023 IEEE 34th International Symposium on Software Reliability Engineering, ISSRE, 2023, pp. 742–751, <http://dx.doi.org/10.1109/ISSRE59848.2023.00048>.
- [46] A. Prokopec, A. Rosà, D. Leopoldseder, G. Duboscq, P. Tůma, M. Studener, L. Bulej, Y. Zheng, A. Villazón, D. Simon, T. Würthinger, W. Binder, Renaissance: benchmarking suite for parallel applications on the JVM, in: Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, in: PLDI 2019, Association for Computing Machinery, New York, NY, USA, 2019, pp. 31–47, <http://dx.doi.org/10.1145/3314221.3314637>.
- [47] A. Peternier, D. Bonetta, W. Binder, C. Pautasso, Tool demonstration: Overseer - Low-level hardware monitoring and management for Java, in: 9th International Conference on the Principles and Practice of Programming in Java, PPPJ, 2011, pp. 143–146, <http://dx.doi.org/10.1145/2093157.2093179>.
- [48] A. Rosà, W. Binder, Optimizing type-specific instrumentation on the JVM with reflective supertype information, *J. Vis. Lang. Comput.* 49 (2018) 29–45, <http://dx.doi.org/10.1016/j.jvlc.2018.10.007>.
- [49] L. Gu, G. Bai, Y. Guo, X. Chen, H. Mei, Security model oriented attestation on dynamically reconfigurable component-based systems, *J. Netw. Comput. Appl.* 35 (3) (2012) 974–981, <http://dx.doi.org/10.1016/j.jnca.2011.03.014>.
- [50] O. Morariu, C. Morariu, T. Borangiu, Transparent real time monitoring for multi-tenant J2EE applications, *Control Eng. Appl. Inform.* 15 (4) (2013) 37–46.
- [51] K. Chapman, A.L. Hosking, J.E.B. Moss, T. Richards, Closed and open nested atomic actions for Java: language design and prototype implementation, in: Proceedings of the 2014 International Conference on Principles and Practices of Programming on the Java Platform: Virtual Machines, Languages, and Tools, PPPJ '14, Association for Computing Machinery, New York, NY, USA, 2014, pp. 169–180, <http://dx.doi.org/10.1145/2647508.2647525>.
- [52] E. Bergen, S. Edlich, Post-debugging in large scale big data analytic systems, in: *Lecture Notes in Informatics (LNI) - Gesellschaft Fur Informatik, GI, Vol. 266*, 2017, pp. 65–72.
- [53] Y. Wang, K.B. Kent, A region-based approach to pipeline parallelism in Java programs on multicores, in: 2017 25th Euromicro International Conference on Parallel, Distributed and Network-Based Processing, 2017, pp. 124–131, <http://dx.doi.org/10.1109/PDP.2017.69>.
- [54] J. Frenzel, K. Feldhoff, R. Jäkel, R. Müller-Pfefferkorn, Tracing of multi-threaded java applications in Score-P using bytecode instrumentation, in: C. Trinitis, T. Pionteck (Eds.), *ARCS 2018 — 31st GI/ITG International Conference on Architecture of Computing Systems, Workshop Proceedings*, VDE Verlag, Berlin, 2018, pp. 51–59.
- [55] B. Kaddour, J. Quinqueton, Proposal for compilation techniques of monitoring tasks to improve applications management performance, in: International Conference on Networking and Services, ICNS, 2006, pp. 33–38, <http://dx.doi.org/10.1109/ICNS.2006.89>.
- [56] B. Kaddour, J. Quinqueton, Using aspects and compilation techniques to dynamically manage applications, in: Proceedings of the 2006 International Conference on Security and Management, SAM'06, Las Vegas, NV, USA, 2006, URL: <https://hal.science/lirmm-00370608>.
- [57] A.D. Silva, A. Gonzalez-Calero, J.F. Martinez, L. Lopez, A.B. Garcia, V. Hernandez, Design and implementation of a Java fault injector for exhaustif SWIFI tool, in: 2009 Fourth International Conference on Dependability of Computer Systems, 2009, pp. 77–83, <http://dx.doi.org/10.1109/DepCoS-RELCOMEX.2009.27>.
- [58] C. Sun, J. Ren, X. Zheng, Y. Wei, X. Zheng, JVMTI-based model enforcement on Java platform for model-carrying code, in: Proceedings of the 4th International Conference on Ubiquitous Information Technologies & Applications, 2009, pp. 1–5, <http://dx.doi.org/10.1109/ICUT.2009.5405715>.
- [59] L.-M. Fei, X.-J. Zheng, Z.-C. Yang, P.-X. Liu, JVMTI-based implementation of security policies enforcement on java platform, in: 2010 3rd International Conference on Advanced Computer Theory and Engineering, Vol. 6, 2010, pp. V6210–V6213, <http://dx.doi.org/10.1109/ICACTE.2010.5579382>.
- [60] N.P. Ricci, S.Z. Guyer, J.E.B. Moss, Elephant tracks: Portable production of complete and precise GC traces, 2013, pp. 109–118, <http://dx.doi.org/10.1145/2464157.2466484>.
- [61] J. Dai, M. Morisio, J.F. Mejia Bernal, A seamless services migration framework with JVM tool interface, in: 5th International Conference on Computer Science and Education, 2010, pp. 106–110, <http://dx.doi.org/10.1109/ICCSE.2010.5593685>.
- [62] N.P. Ricci, S.Z. Guyer, J.E.B. Moss, Tool demonstration: Elephant tracks-generating program traces with object death records, in: 9th International Conference on the Principles and Practice of Programming in Java, PPPJ, 2011, pp. 139–142, <http://dx.doi.org/10.1145/2093157.2093178>.
- [63] A. Rosà, E. Rosales, W. Binder, Accurate reification of complete supertype information for dynamic analysis on the JVM, in: Proceedings of the 16th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences, in: GPCE 2017, Association for Computing Machinery, New York, NY, USA, 2017, pp. 104–116, <http://dx.doi.org/10.1145/3136040.3136061>.
- [64] A. Schögenhumer, P. Hofer, D. Gniedt, H. Mössenböck, Efficient sampling-based lock contention profiling for Java, in: 2017 ACM/SPEC International Conference on Performance Engineering, ICPE, 2017, pp. 331–334, <http://dx.doi.org/10.1145/3030207.3030234>.
- [65] R. Liscano, A. Ahmed, J. Robertson, A. Azim, V. Sundaresan, Y.-K. Chan, A lock contention classifier based on Java lock contention anti-patterns, in: 2023 International Conference on Machine Learning and Applications, ICMLA, 2023, pp. 1106–1113, <http://dx.doi.org/10.1109/ICMLA58977.2023.00165>.
- [66] A. Pantaleev, A. Rountev, Identifying data transfer objects in EJB applications, in: Fifth International Workshop on Dynamic Analysis, WODA '07, 2007, <http://dx.doi.org/10.1109/WODA.2007.6>, 5–5.
- [67] A. Rosà, Y. Zheng, H. Sun, O. Javed, W. Binder, Adaptable runtime monitoring for the Java virtual machine, in: T. Margaria, B. Steffen (Eds.), *Leveraging Applications of Formal Methods, Verification and Validation: Discussion, Dissemination, Applications*, Springer International Publishing, Cham, 2016, pp. 531–546.
- [68] M. Basso, E. Rosales, F. Schiavio, A. Rosà, W. Binder, Accurate fork-join profiling on the Java virtual machine, in: J. Cano, P. Trinder (Eds.), *Euro-Par 2022: Parallel Processing — 28th International Conference on Parallel and Distributed Computing*, Glasgow, UK, August 22–26, 2022, Proceedings, in: *Lecture Notes in Computer Science*, vol. 13440, Springer International Publishing, Cham, 2022, pp. 35–50, http://dx.doi.org/10.1007/978-3-031-12597-3_3.
- [69] E. Rosales, A. Rosà, M. Basso, A. Villazón, A. Orellana, Á. Zenteno, J. Rivero, W. Binder, Characterizing Java streams in the wild, in: 2022 26th International Conference on Engineering of Complex Computer Systems, ICECCS, 2022, pp. 143–152, <http://dx.doi.org/10.1109/ICECCS54210.2022.00025>.
- [70] G. Jacques-Silva, R. Jung Drebes, J. Gerchman, T. Weber, FIONA: a fault injector for dependability evaluation of Java-based network applications, in: Third IEEE International Symposium on Network Computing and Applications, NCA, 2004, pp. 303–308, <http://dx.doi.org/10.1109/NCA.2004.1347791>.
- [71] A. Knüpfer, C. Rössel, D.a. Mey, S. Biersdorff, K. Diethelm, D. Eschweiler, M. Geimer, M. Gerndt, D. Lorenz, A. Malony, W.E. Nagel, Y. Oleynik, P. Philippen, P. Saviankou, D. Schmidl, S. Shende, R. Tschüter, M. Wagner, B. Wesarg, F. Wolf, Score-P: A joint performance measurement run-time infrastructure for periscope,scalasca, TAU, and vampir, in: H. Brunst, M.S. Müller, W.E. Nagel, M.M. Resch (Eds.), *Tools for High Performance Computing 2011*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2012, pp. 79–91, http://dx.doi.org/10.1007/978-3-642-31476-6_7.
- [72] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, Springer, Berlin, Heidelberg, 2012, <http://dx.doi.org/10.1007/978-3-642-29044-2>.